SE-MPPI: Coordinating Online Local-Minima Escape with Control-Barrier-Function Safety in a Nav2-Native Controller

Jungmo Kang

***Abstract*—The Model Predictive Path Integral (MPPI) controller shipped with ROS 2 Nav2 is a widely used deployable local controller for mobile robots, yet in narrow, crowded, dynamic spaces it can stall in local minima, and its critics avoid collisions only through soft costs, with no formal safety guarantee. Methods exist for each weakness, and several works outside Nav2 address both together, but we know of no public Nav2 controller plugin that combines local-minima escape with a barrier-based safety filter. This system paper presents Safe-Escape MPPI (SE-MPPI), a Nav2 controller plugin that integrates online local-minima detection-and-escape for MPPI with a control-barrier-function (CBF) safety filter for tracked dynamic obstacles. A first version of the plugin reached the goal in none of its 44 valid live trials on a correct build; we report the four defects that we identify as the cause (one inferred, not logged) and the repaired controller. Its escape carries four additions to the advised design: two of them, avoid lists and a movement watchdog, were needed for the development sweeps of a family of pockets to pass (ablated jointly; the ablation is confounded with position in the sweep), and two are safeguards for other configurations. We disclose all four, with a joint ablation of the first two. In a pre-registered live campaign (one post hoc amendment to its crash-stop rule, after which all s4 SE-MPPI trials ran), SE-MPPI without recoveries reached the goal in 5 of 6 on s1 (Wilson 95% interval $[0.44, 0.97]$) and 5 of 6 on the held-out s4, which lies outside the development family; stock without recoveries in 0 of 6 on each; with Nav2's default recovery tree, on s1, stock 0 of 6 and SE-MPPI 3 of 6, on one simulated host (Fisher exact $p = 0.0152$ for each of the first two contrasts and $p = 0.1818$ for the third, which is descriptive). s1 is the development scenario; s4 carries the claim. A third pocket, s3, that has a near mirror twin in the development family gave 4 of 6 against 0 of 6 ($p = 0.0606$, not significant). We make no claim beyond one simulated host, six repetitions per cell, and no real robot. In a randomized, paired $1{,}200$-trial benchmark on a Python 2D mirror that transcribes the shipped algorithm, not on the Nav2 controller itself, escape raises success on the static U-trap family from 0% to 80% (Wilson 95% interval $[67, 89]$); without the four additions the same benchmark gives 58%. The escape–safety coordination matches an independent escape+CBF stack there, a null result, so we claim it only as a mechanism. By a standard argument for time-varying CBF gains, the gain schedule preserves forward invariance of the barrier's safe set, a conditional guarantee that covers only a look-ahead point and the tracked dynamic obstacles, assumes that the barrier inequality holds between control updates, and requires a slack-free quadratic program.**

***Index Terms*—Model predictive control, control barrier functions, collision avoidance, local-minima escape, motion and path planning, autonomous navigation, ROS 2 Nav2.**

J. Kang is an independent researcher based in Seoul, Republic of Korea (e-mail: kangjmo91@gmail.com).

I. Introduction

Autonomous mobile robots increasingly operate in cluttered, dynamic indoor environments – warehouses, hospitals, service floors – where the local controller must be simultaneously *fast*, *non-conservative*, and *safe*. Within the ROS 2 Nav2 ecosystem, the MPPI controller (`nav2_mppi_controller`) [1] is a widely used high-performance option: it supports differential, omnidirectional, and Ackermann robots and works by sampling `batch_size` rollouts, scoring them with a bank of critics, and combining them with an information-theoretic softmax [2].

Two weaknesses, however, recur exactly in the environments that matter most:

1) **Local-minima entrapment.** A finite prediction horizon (by default $56 \times 0.05\,\mathrm{s} \approx 2.8\,\mathrm{s}$) and zero-mean Gaussian sampling cause the robot to stall or oscillate in non-convex traps. Within the MPPI critic set, Nav2's mitigations (`PreferForwardCritic`, `TwirlingCritic`) are *always-on* cost shaping that neither *detects* entrapment nor steers an escape. At the stack level, Nav2 does detect a stall: the controller server's progress checker reports a failure when the robot has not moved a set distance within a set time, and the default behavior tree then clears the costmaps and runs generic recoveries such as spinning, waiting, and backing up [1], [3], [4]. These recoveries are not directed toward an opening. In the MPPI literature, one line of work reshapes the sampling distribution all the time, in part to reduce entrapment [5], [6], [7] and in part for other aims [8], [9]; another adds guidance: GP-MPPI recommends a free-space subgoal from a learned navigability surface [10], and repulsive-potential augmentation adds an artificial potential field to the MPPI cost [11], which DRPA-MPPI switches on only when it detects entrapment [12]. None of these carries a safety certificate.
2) **No formal safety.** In Nav2 MPPI, avoidance is encoded as soft costs (`ObstaclesCritic`, `CostCritic`); there is no forward-invariance guarantee, and crucially **no prediction of dynamic-obstacle motion** – the costmap is a present-time snapshot and `collision_monitor` is purely reactive. CBF–MPPI fusions supply safety [13], [14], [15], [16], [17], but

none of them detects entrapment or adds an escape mechanism (GS-MPPI counters the myopic behavior often associated with CBFs through long-term performance terms), and none ships as a Nav2 controller. Among CBF integrations with Nav2 that we could verify, the Nav2 documentation describes inserting a third-party product, the 3Laws Supervisor, which its vendor describes as based on CBFs, into the command chain after the controller [18], [19]; it is a separate node rather than a controller plugin, and neither source describes a mechanism for escaping local minima.

These weaknesses also interact: being conservative for safety invites entrapment, and escaping aggressively invites collision. Several works outside Nav2 and MPPI address both at once, among them model predictive control that escapes local minima while keeping a safe backup trajectory [20], gap-based local planners with provable collision-free properties [21], [22], [23], and modulated CBF filters [24] that reduce or eliminate, among moving obstacles, the spurious equilibria that CBF quadratic programs (QPs) can introduce [25], with concurrent extensions [26], [27]. In SE-MPPI, the entrapment detector is driven by progress along a per-task anchor path and runs inside the controller rather than at the stack level; Appendix B keeps a standalone 2D illustration of the design stage. This is a system contribution, and we state it as such.

Contributions.

- **C1 (system).** A public **Nav2 local-controller plugin** that combines online local-minima detection-and-escape with a CBF safety filter for tracked dynamic obstacles, packaged as a `nav2_core::Controller` plus a pluginlib MPPI critic. Each ingredient has precedent: gated repulsion in DRPA-MPPI [12], free-space subgoal and gap guidance in GP-MPPI [10] and gap-based planners [21], [22], CBF filtering of MPPI commands [15], and a supervisor for Nav2 that its vendor describes as CBF-based [18], [19]. What is new is their integration inside one Nav2 controller; to our knowledge no public controller plugin already provides it.
- **C2 (repair and live evidence).** The first version of the plugin did not reach the goal in a live simulated stack. We name the four defects behind that (Sec. VI-A), describe the repaired controller that ships (Sec. IV), disclose the four additions to the advised escape (two needed together for the development sweeps to pass, ablated jointly; two safeguards for other configurations), and test it in a pre-registered live campaign on one simulated host, with one post hoc amendment to its crash-stop rule after which all s4 SE-MPPI trials ran (Sec. VI-E). The campaign supports a narrow claim: the repaired controller without Nav2 recoveries reached the goal in 5 of 6 trials on the development pocket and in 5 of 6 on a held-out pocket outside the development family, where stock MPPI reached it in none.
- **C3 (algorithm).** An **entrapment-gated gain schedule**: a rule that raises the CBF class-$\mathcal{K}$ gain on detected entrapment and drops it back on an imminent time-to-collision (TTC). Online adaptation of CBF gains is established, including selecting class-$\mathcal{K}$ parameters to reduce predicted deadlock [28] and letting MPPI optimize a barrier-rate parameter [14]; our rule is a simpler, rule-based two-level instance whose specific elements are its trigger and its TTC override. Its invariance property (Sec. IV-F) is a known result for time-varying class-$\mathcal{K}$ terms [29], [30], which we restate as a conditional lemma: it covers the look-ahead point and the tracked dynamic obstacles, assumes the barrier inequality holds between samples, and requires a slack-free QP. In our 2D benchmark the schedule is empirically indistinguishable from an independent escape+CBF stack (Sec. VI-D1), and the live campaign does not test it: its pockets contain no moving body (Sec. VI-E1 reports the phantom obstacles and forced stops that occurred nevertheless, which were not a designed test of the coordination).
- **C4 (dynamic safety).** Integration of a look-ahead-point CBF safety filter for a differential-drive robot, fed by a costmap-based dynamic-obstacle tracker, as a QP at the controller's output. The live campaign ran it with the occupancy-based static/dynamic classification, the horizon prediction and the conformal margin at their code defaults (on); the 2D mirror has none of the three (Sec. IV-G).
- **C5 (reproducibility).** An open Apache-2.0 implementation,[1] a Python 2D mirror of the shipped algorithm with a randomized 1,200-trial benchmark and paired significance tests, a headless closed-loop development test with a written family rule, and the frozen, hash-checked pre-registration of the live campaign with its amendments.

We are deliberately conservative about novelty and about what we claim empirically. Fusing CBFs with MPPI is *not* new, and neither is gated escape, online adaptation of the CBF gain, or treating escape and safety together; the paper is best read as a system report on putting these pieces into one Nav2 controller, on what went wrong the first time, and on a small pre-registered live test of the repaired version. The live claim is limited to what the pre-registered branch wording allows (Sec. VI-E): counts out of six, Fisher exact p values and Wilson intervals, on one simulated host. In the 2D mirror of the shipped algorithm the escape layer is the measured effect on the trap families (Sec. VI-D). The escape–safety coordination (C3) has the conditional invariance property of Sec. IV-F yet, in the regime we could test, is empirically indistinguishable from an independent escape+CBF stack; we report this null openly. We ran no real robot, no third-party escape baseline, and no full-scale physics benchmark (Sec. VII).

II. Related Work

MPPI and local controllers. Nav2 MPPI [1], [2], TEB [31], DWB (a Nav2 DWA-family planner) [32],

[1] Code, benchmark scripts and the campaign pre-registration: https://github.com/kjungmo/se-mppi-nav2 (Apache-2.0); the raw trial data of campaign 2 are the archive `experiments/campaign2/raw_trials.tar.gz`; see the Appendix for the commands. At the time of writing, the tag, the commits and the archive cited in this paper exist only in the author's local repository. A release ref will be published before submission; the commit of the local tag `campaign2-controller` (`6a50723`) is the provenance.

and Regulated Pure Pursuit [33] are the standard deployable local planners. MPPI's critic-plugin interface (`mppi::critics::CriticFunction`) exposes per-rollout trajectories, the reference path, the goal, collision flags, and the furthest-reached path index – i.e., the signals needed for entrapment detection are already available, and a custom critic can be inserted without forking the optimizer. Around the controller, Nav2 already handles stalls at the stack level: the controller server runs a progress checker, by default `SimpleProgressChecker`, which reports a failure when the robot has not moved a set radius within a set time allowance, and the default behavior tree answers with costmap clearing, spinning, waiting, and backing up [3], [4]. SE-MPPI's detector differs in that it runs inside the controller on progress along the path and steers the escape instead of triggering generic recoveries; the two mechanisms can coexist.

Local-minima escape. Most MPPI variants that bear on local minima are *always-on* changes of the sampling distribution: log-MPPI [5] and Biased-MPPI [6] name entrapment as a motivation and learned sampling distributions [7] mention poor local optima, whereas Tsallis-divergence MPC [8] and SVG-MPPI [9] change the distribution for other reasons. Others add guidance. GP-MPPI learns a sparse-Gaussian-process model of the navigable space around the robot and recommends a free-space subgoal to the local MPPI planner, which lets the robot escape local minima without a global map [10]. Fuke et al. add a repulsive artificial potential to MPPI to avoid local minima without global path guidance [11], and their DRPA-MPPI detects potential entrapment on the predicted trajectories and only then switches to a cost with repulsion away from the local minimum [12]. DRPA-MPPI is the closest *detect-and-switch* method and GP-MPPI the closest subgoal method; neither has a CBF or other formal safety layer, and neither is Nav2-integrated. SE-MPPI combines both ideas in a simpler form, a path-progress stall detector gating a classical repulsive potential [34] and a ray-cast gap bearing, and its safety filter runs alongside the escape rather than gating it. Gap-based local planners are closer still to the overall architecture. Potential Gap combines gap-based local navigation with artificial potential fields and restores provable collision-free properties for realistic robot models [21]; Safer Gap adds nonlinear model predictive control with keyhole zeroing-barrier-function constraints and a point-wise optimization-based safety filter as the final layer [22]; and Dynamic Gap extends provably collision-free gap-based navigation to dynamic environments by modeling how gaps evolve over time [23]. Relative to this line, SE-MPPI uses a gap only as a gated escape heuristic inside MPPI, and its barrier covers only tracked dynamic obstacles.

Escape and safety together. Several works treat entrapment and safety jointly. Soloperto et al. optimize an exploration trajectory together with a safe backup trajectory in a model predictive controller and prove convergence, feasibility, and constraint satisfaction under partially unknown constraints [20]. The modulated CBF-QP of Xue and Figueroa reduces or eliminates, in dynamic environments, the spurious equilibria that CBF-QP filters create [25], and was tested on Ridgeback and Fetch robots [24]. Concurrent work adds learned predictive barriers for moving concave obstacles to this filter [26], and pairs a mixed-integer MPC planner with a Minkowski-difference CBF filter to mitigate the local-minimum behavior of reactive CBF navigation in U-shaped and maze-like environments [27]. These methods mitigate or avoid minima inside the planner or the barrier itself, whereas SE-MPPI leaves the barrier unchanged and escapes through the MPPI cost. None of them is packaged as a Nav2 controller.

CBF $\times$ MPPI safety. Three fusion styles exist as research code: CBF conditions in the rollout cost or the sampling (Shield-MPPI [13], which adds a local repair step, and BR-MPPI [14], which imposes the CBF condition as an equality with a parametric linear class-$\mathcal{K}$ function, augments the state with its parameter, and lets MPPI optimize the parameter's rate as an extra input), an output CBF-QP filter on the MPPI command (SCBF-MPPI [15], built on the CBFKit toolbox [35]), and per-rollout safety filtering that makes every sample provably safe (GS-MPPI [17], DualGuard [16], which also filters the final command). For dynamic obstacles, DPCBF [36] uses a parabolic safe set that is less conservative than collision-cone barriers but is QP-only and not embedded in MPPI. Our filter builds on the classical CBF-QP [37], [38]. None of these works ships as a Nav2 controller (CBFKit connects to ROS through its own wrapper module, not through Nav2's controller interface), and none detects entrapment or adds an escape mechanism, although GS-MPPI counters the myopic behavior often associated with CBFs through long-term performance terms [17]. Among Nav2 integrations of a CBF that we could verify, the official documentation describes inserting the third-party 3Laws Supervisor, which its vendor describes as based on CBFs, between the velocity smoother and the collision monitor [18], [19]; it acts on the controller's output as a separate node, and neither source describes a local-minimum escape mechanism. SE-MPPI instead places its filter inside the controller and couples it to the escape layer through its gain.

Online adaptation of the CBF gain. Time-varying and adapted class-$\mathcal{K}$ terms are well studied. Adaptive CBFs multiply the class-$\mathcal{K}$ functions by time-varying penalty functions and prove forward invariance of the safe set [29]; optimal-decay CBF-QPs optimize the decay rate point-wise in time to keep the QP feasible [39]; and rate-tunable CBFs adapt the class-$\mathcal{K}$ functions online and derive conditions under which multiple constraints can share the control input [30]. Closest to our trigger, Kim, Kee, and Panagou adapt the parameters of input-constrained CBFs online with a learned ensemble model and select, among the candidates that pass their uncertainty and risk checks, the class-$\mathcal{K}$ parameters with the minimum predicted deadlock time [28]; BR-MPPI lets MPPI optimize the rate of the class-$\mathcal{K}$ parameter as an extra control input [14]. Concurrent work switches nominal objectives when CBF and control-Lyapunov constraints conflict, to reduce slowdowns and deadlocks [40], and schedules the gain of a frontier barrier on an occupancy grid by how well the map is explored [41]. SE-MPPI's coordinator is a rule-based, two-level instance of this idea, triggered by the entrapment flag and overridden by time-to-collision, and its invariance property (Sec. IV-F) follows the same argument as the relative-degree-one case of

the adaptive-CBF result [29, Thm. 3], there stated for Lipschitz controllers.

Conformal prediction for safe control (background, not claimed). The released controller carries an online conformal margin that inflates the CBF effective radius under prediction uncertainty; this is an implementation detail deferred to follow-up work (Sec. VII, SE-Predict) and is *not* a contribution of this paper. That margin draws on the established line of conformal-prediction-for-safe-control work [42], [43], [44], [45], [46]; the claims of this paper (C1–C4) stand without it. The live campaign ran with the margin enabled (Sec. IV-G).

Positioning. Table I places the reviewed systems against four factors: **(a)** shipping as a Nav2 controller plugin, **(b)** online local-minimum escape or avoidance, **(c)** an explicit runtime safety constraint, filter or backup, and **(d)** online adaptation of the class-$\mathcal{K}$ gain. No reviewed system holds all four, but several hold two, fully or in part: stock Nav2 pairs its controller plugins with stack-level stall recovery, gap-based planners, backup-trajectory MPC and modulated CBFs combine local-minimum handling with a safety layer, and adaptive-gain CBFs and BR-MPPI pair a barrier with gain adaptation, with deadlock-aware adaptation also holding the second factor in part. The delta of SE-MPPI is therefore the conjunction inside one Nav2 controller plugin, not any single element. Its own marks are not all full: its barrier covers only tracked dynamic obstacles, and its escape is demonstrated live only in simulation, on one host, with six repetitions per cell, and in the Python 2D mirror. Its gain rule differs from prior gain adaptation mainly in its trigger, and, as Sec. VI-D1 shows, we could not measure an outcome benefit from it.

III. Problem Formulation

Consider a differential-drive robot with pose (x, y, θ) and control $u = (v, \omega)$, tracking a global path σ produced by a Nav2 global planner, amid static structure and a set $\mathcal{O}$ of dynamic obstacles o with position p_o, radius R_o, and (estimated) velocity v_o. Let r be the radius of the disc that stands for the robot in the barrier (by default the circumscribed radius of the costmap footprint) and m a safety margin. We want a control policy that (G1) makes monotone progress along σ – i.e., does not stall in a local minimum – and (G2) keeps the robot clear of the obstacles in $\mathcal{O}$ through a barrier whose forward invariance holds under stated conditions, while (G3) running as a Nav2 controller at ≥ 10 Hz on CPU.

The tension: G1 may require maneuvers that approach obstacles (to round a trap), whereas a naive safety filter for G2 forbids exactly those maneuvers. SE-MPPI's coordinator (Sec. IV-F) couples the escape layer for G1 and the barrier for G2 through the barrier gain.

IV. Method: SE-MPPI

This section describes the controller that ships, tagged `campaign2-controller`. The first version of the plugin differed in four points that Sec. VI-A names; the escape described here also contains four additions to the design we advised ourselves, which Sec. IV-D lists.

A. Overview

SE-MPPI subclasses the stock `MPPIController`, reusing its optimizer for the nominal command, then post-processes that command each cycle:

1) **Entrapment detection** from progress along a per-task anchor path (single source of truth, shared with the critic).
2) **Escape**: while entrapped, the controller hands MPPI a short escape path to a waypoint in an opening, and an `EscapeCritic` adds a distance-field repulsion cost to the rollouts.
3) **Dynamic-obstacle tracking** from the local costmap.
4) **Coordination**: resolve the CBF gain α from entrapment and TTC.
5) **CBF safety filter**: project the nominal (v, ω) onto the CBF-safe set via a small QP.

The escape and safety layers thus act at complementary points – sampling-time shaping of the path and the costs, and output-time projection – unified by a shared entrapment signal and the coordinated gain.

Fig. 1 summarizes the resulting data flow.

B. Entrapment detection (single source of truth)

Progress on an anchor path. Progress is the arc length along a per-task *anchor path*, the first plan of the task. The nearest-index progress of the first version, re-anchored on every replan, let one path cell of motion along a fresh replan count as progress; Nav2's behavior tree replans every second, so this ended every escape within seconds (Sec. VI-A). A progress event now needs 0.10 m of arc length beyond the best so far. A replan to the same goal (same frame, and an endpoint within two cells of the local costmap, 0.10 m here, of the previous endpoint) is only a *candidate*: it becomes the anchor, and counts as an event, once the signed advance along the chain of candidates reaches 0.5 m. Entrapment is declared when no event occurs for $W = 30$ control cycles (3 s at 10 Hz; in the live campaign the loop was paced by the wall clock, Sec. VI-E) and is suppressed within a goal tolerance of 0.5 m. The window counts control cycles, not seconds: at the 20 Hz of the released parameter file the same 30 cycles are 1.5 s. The movement watchdog (3 s) and the re-selection period (15 s) of Sec. IV-C are also converted to cycles at the configured controller frequency, so all three stretch in wall time when the loop misses its rate. The escape is decided at the start of a cycle from the detector state at the end of the previous cycle, a lag of one cycle. The controller publishes the detector state to the `EscapeCritic` through a shared atomic flag in a registry keyed by the controller name. The critic scores the rollouts before the detector of the current cycle runs, so the critic and the escape path act on the previous cycle's state, while the coordinator uses the state of the current cycle. Escape and safety therefore agree on whether the robot is trapped up to one cycle: on the cycle in which entrapment begins or ends, the gain α already follows the new state and the repulsion and the escape path still follow the old one. The mirror has the same split.

TABLE I

POSITIONING. **(A)** SHIPS AS A NAV2 CONTROLLER PLUGIN; **(B)** ONLINE LOCAL-MINIMUM ESCAPE OR AVOIDANCE; **(C)** AN EXPLICIT SAFETY CONSTRAINT, FILTER OR BACKUP ENFORCED AT RUN TIME (FOR DUALGUARD, HAMILTON–JACOBI REACHABILITY; FOR SOLOPERTO ET AL., A SAFE BACKUP TRAJECTORY; FOR THE GAP-BASED PLANNERS, A SAFETY PROJECTION OPERATOR ON THE COMMAND AND, IN DYNAMIC GAP AND SAFER GAP, ZEROING BARRIER FUNCTIONS); **(D)** ONLINE ADAPTATION OF THE CBF CLASS-$\mathcal{K}$ GAIN. ✓ = HOLDS; ○ = HOLDS IN PART: STOCK NAV2 DETECTS STALLS AT THE STACK LEVEL AND RUNS GENERIC RECOVERIES RATHER THAN A STEERED ESCAPE; THE 3LAWS SUPERVISOR IS INTEGRATED WITH NAV2 AS A SEPARATE NODE AFTER THE CONTROLLER, NOT AS A CONTROLLER PLUGIN; THE GAP-BASED PLANNERS GUIDE THE ROBOT THROUGH FREE-SPACE GAPS RATHER THAN ESCAPING A DETECTED MINIMUM; KIM ET AL. REDUCE PREDICTED DEADLOCK THROUGH CLASS-$\mathcal{K}$ PARAMETER SELECTION RATHER THAN ESCAPING A DETECTED MINIMUM. FOR THE 3LAWS SUPERVISOR, (B) IS MARKED ABSENT BECAUSE NEITHER CITED SOURCE DESCRIBES A LOCAL-MINIMUM ESCAPE MECHANISM. †CONCURRENT WORK (POSTED IN 2026). FOR SE-MPPI, (C) IS MARKED IN PART BECAUSE ITS BARRIER COVERS ONLY TRACKED DYNAMIC OBSTACLES, AT A LOOK-AHEAD POINT, AND ITS INVARIANCE ARGUMENT IS CONDITIONAL (SEC. IV-F); STATIC STRUCTURE IS HANDLED ONLY BY MPPI COSTS. ITS (B) IS DEMONSTRATED IN A SIMULATED LIVE NAV2 STACK ON ONE HOST (SEC. VI-E) AND IN THE PYTHON 2D MIRROR, NOT ON A REAL ROBOT. NO REVIEWED SYSTEM HOLDS ALL FOUR; SEVERAL HOLD TWO. THE REVIEW BEHIND THIS TABLE (JUNE 2026, EXTENDED IN SEPTEMBER 2026) COVERED THE NAV2 MPPI CRITIC ECOSYSTEM, THE NAV2 DOCUMENTATION, THE PUBLIC ISSUE TRACKERS OF THESE PROJECTS, AND THE WORKS CITED HERE; IT IS NOT EXHAUSTIVE.

System	(a)	(b)	(c)	(d)
Nav2 MPPI with progress checker and recoveries [1], [3], [4]	✓	○	–	–
3Laws Supervisor in Nav2 [18], [19]	○	–	✓	–
DRPA-MPPI, GP-MPPI [12], [10]	–	✓	–	–
Dynamic Gap, Potential Gap, Safer Gap [23], [21], [22]	–	○	✓	–
Backup-trajectory MPC [20]	–	✓	✓	–
MILP-MPC with CBF†; MCBF-QP; predictive MCBF† [27], [24], [26]	–	✓	✓	–
DualGuard, SCBF-MPPI, GS-MPPI, Shield-MPPI [16], [15], [17], [13]	–	–	✓	–
CBFKit, DPCBF [35], [36]	–	–	✓	–
BR-MPPI [14]	–	–	✓	✓
Rate-tunable, adaptive, optimal-decay CBFs [30], [29], [39]	–	–	✓	✓
Deadlock-aware parameter adaptation [28]	–	○	✓	✓
SE-MPPI (ours)	✓	✓	○	✓

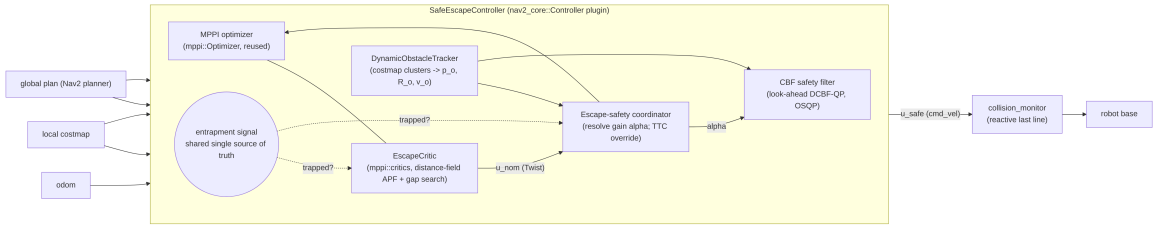


Fig. 1. SE-MPPI architecture / data flow, as intended. Global plan, local costmap, and odometry feed the `SafeEscapeController` plugin: the reused MPPI optimizer (with the `EscapeCritic`) emits the nominal command, a shared entrapment signal drives both the critic and the escape–safety coordinator, the coordinator resolves the CBF gain α (with TTC override), and the CBF safety filter – fed by the `DynamicObstacleTracker` – projects the command to `cmd_vel`. The intended chain ends in Nav2's `collision_monitor` as a final reactive layer; in the live campaign the only `collision_monitor` polygon was disabled, so that stage was not active (Sec. VII). The figure predates the repaired escape: the critic's gap search that it labels is off in the shipped configuration (`gap_weight` 0); the opening selection, which now produces the escape, runs in the controller, and its hand-off of an escape path to the optimizer (Sec. IV-C) is not drawn. Nav2's velocity smoother, which sits between the controller and the base in the campaign, is not drawn either.

No valid control. When every rollout collides, the optimizer throws `NoValidControl`. The controller catches it, runs the progress update and the detector as on a normal cycle, and rethrows. This update acts only if the controller server tolerates the failure. With Nav2's default `failure_tolerance` of 0 the server rethrows, publishes zero velocity and calls `reset()` on every controller, which clears the detector in the same cycle; the live campaign of Sec. VI-E runs with this default, so the update did not act there (Sec. VII).

Reverse speed. The parameter file keeps `vx_min` at -0.35 m/s. With `vx_min` > 0 the robot cannot stop or back off, so the escape cannot act; the controller logs a warning, and the campaign driver refuses such a parameter file.

C. Escape (detect-and-switch)

While the robot is entrapped, the controller applies three rules. Table II lists the values.

(1) Opening selection. The controller casts 36 rays over the local costmap out to a sensing horizon of 2.0 m (or the costmap edge). Among the rays that reach the horizon, whose free length (to the first cell with cost at least 253, the inscribed band, not the wall, or to the first unknown cell) is at least 0.6 m, and that do not lead to an avoided place (Sec. IV-D), it takes the ray closest to the goal bearing. If there is none, it takes the longest ray whose free length is at least 0.6 m and that does not lead to an avoided place. Both uses of the 0.6 m threshold are part of addition (d) (Sec. IV-D); the advice had no threshold on the longest-ray fallback. If excluding rays leaves nothing, it repeats the choice without the avoid list. A hysteresis of 0.5 rad keeps the previous bearing unless a new ray is closer to the goal bearing by more than that. The horizon test matters: from the mouth of a pocket the ray back into the pocket is longer than the rays along the walls, so a plain longest-ray rule re-enters the pocket.

(2) Escape path. The waypoint lies on the selected bearing

at $\min(c - r - 0.05,\ 2.5)$ m, with c the free length of the ray and r the radius of the CBF disc (`se_cbf_robot_radius`; its default 0 takes the circumscribed radius that the costmap reports for the footprint). The campaign file does not set it, and every live launch logged `robot_radius=0.234` at configure, for a footprint built from `robot_radius` 0.22 m, so $r = 0.234$ m live. A deployer who sets the CBF radius therefore also moves the escape waypoint. The waypoint is taken only if that distance is at least 0.2 m. Four values are compiled constants, not YAML values: the floor of 0.2 m, the margin of 0.05 m in the waypoint distance, the size 6 of the failed list (below) and the same-goal test of Sec. IV-B (two costmap cells). If no ray is usable when the escape starts, MPPI keeps the behavior-tree path; if none is usable at a re-selection, MPPI keeps the previous escape path. In both cases the selection runs again in every cycle, and the repulsion stays on while the robot is entrapped. The mirror does the same. The cap of 2.5 m is inactive while the sensing horizon 2.0 m is at most $2.5 + r + 0.05 = 2.784$ m, as here: the longest hop is $2.0 - 0.234 - 0.05 = 1.716$ m. The controller builds a path from the robot to the waypoint and hands it to MPPI through `MPPIController::setPlan`, before the optimizer call; the goal, alignment and path-following critics then pull the rollouts toward the waypoint. The path of the behavior tree stays in the controller for progress and for the near-goal test, and MPPI receives it again when the escape ends. Nav2's controller server checks goal arrival against its own copy of the behavior-tree path, so the substitution cannot fake a goal. The path is published on a topic of its own for logging.

(3) Hops. The waypoint stays frozen until the robot is within 0.4 m of it, 15 s have passed, the robot has moved less than 0.15 m in 3 s, or the straight line to it is blocked. Each new selection is a hop. A waypoint that was not reached is remembered as failed for the episode. As the code reads, the waypoint is appended to the failed list in every cycle in which a re-selection fires, also when the selection is then not taken (no usable ray, or a waypoint closer than 0.2 m), so the six entries can hold copies of one waypoint. The mirror has the same quirk. We did not remove the duplicates and did not measure their effect; the campaign ran with them.

End of the escape. Entrapment, and with it the escape and its avoid lists, ends at the first progress event (0.10 m beyond the best arc length on the anchor, or an adoption). The controller then clears the escape state and hands MPPI the behavior-tree path again. All escape memory (anchor path, detector, stall pose, avoid and failed lists, obstacle tracks) lives for one task. A plan that arrives after the control loop has been idle for more than `se_task_idle_threshold` (compiled default 1.0 s, not set in the campaign file) starts a new task, and Jazzy's controller server calls `reset()` at the end of every `FollowPath` goal; both clear this memory. Under a recovery tree, each recovery therefore restarts the escape from scratch. Campaign 2 did not exercise this: no SE trial invoked a recovery or had a progress failure.

Repulsion cost. The `EscapeCritic` keeps the distance-field repulsive cost of the classical form [34], $U = \frac{1}{2}\eta\,(1/d - 1/d_0)^2$ for $d < d_0$, with $\eta = 1$ (the compiled default `apf_eta`; the campaign file does not set it) and $d_0 = 0.6$ m (`apf_influence_dist`). Here d is the Dijkstra distance (8-connected grid) to the nearest cell with cost at least 253 (the inscribed band, which lies one robot radius from the wall, not the wall; unknown space is not an obstacle), clamped below at half a cell (0.025 m). The cost of a rollout is the mean of U over its steps that lie on the map (steps off the map are skipped). It keeps rollouts off the trapping surface during the turn. No weight multiplies it: `repulsion_weight` (2.0 in the parameter file) is unused while `use_apf` is true (its default). The campaign parameter file is frozen, so the comment that would say so is not added there; the released parameter file carries it. The critic's own gap-bearing cost, which rewards rollout endpoints that head toward the selected opening and is a form of the free-space subgoal and gap guidance of GP-MPPI [10] and Potential Gap [21], is off in the shipped configuration (`gap_weight` 0) and kept only for ablation. The first version relied on that cost and selected a sideways opening instead of routing out of the pocket (Sec. VI-A). Because the escape is active only when entrapped, free-space behavior is unchanged (detect-and-switch, not always-on, as in DRPA-MPPI [12]).

TABLE II

ESCAPE PARAMETERS OF THE SHIPPED CONTROLLER (SET IN `EXPERIMENTS/SIM/NAV2_SE_CAMPAIGN2.YAML`, EXCEPT THE FOUR COMPILED CONSTANTS: THE 0.2 M FLOOR, THE 0.05 M WAYPOINT MARGIN, THE FAILED-LIST SIZE 6 AND THE SAME-GOAL TOLERANCE OF TWO COSTMAP CELLS). "ADVISED" MARKS VALUES FIXED BY OUR DESIGN ADVICE BEFORE ANY CODE EXISTED; "ADDITION" MARKS THE RULES AND VALUES OF SEC. IV-D. THE 0.6 M VALUE IS THE ADVISED DEFAULT OF `SE_GAP_MIN_CLEARANCE`; ITS USE AS A THRESHOLD, ON HORIZON RAYS AND ON THE LONGEST-RAY FALLBACK, IS ADDITION (D). (`PREREG.MD`, LINE 147, LISTS ONLY THE USE ON HORIZON CANDIDATES AS AN ADDITION; THE FROZEN FILE IS NOT EDITED.)

Parameter	Value	Origin
stall window	30 cycles (3 s)	advised
progress step / adoption	0.10 m / 0.5 m	advised
rays / horizon	36 / 2.0 m	advised
waypoint cap	2.5 m	advised
re-select radius / period	0.4 m / 15 s	advised
hysteresis	0.5 rad	advised
minimum ray clearance	0.6 m	advised value; threshold: (d)
movement watchdog	3 s / 0.15 m	addition
avoid radius, stall pose	1.0 m	addition
avoid radius, hop origins and failed waypoints; hops	0.6 m; 3	addition
waypoint floor; failed list	0.2 m; 6	addition
`gap_weight`	0	advised

D. Additions to the advised design

The design we advised ourselves before writing code contained the rules above except four additions. We found (a) and (b) needed together for the development sweeps of a family of pockets to pass (Sec. VI-C): the measured ablation below switches them off jointly, and we ran no ablation of either one alone. That ablation is confounded with the position of a geometry in the sweep process: run alone, geometries passed more often without the additions than in the sweeps (below), so it does not show that (a) and (b) are necessary in general. The reason given for each below rests on development

observations, not on a separate measurement. The floor (c) and the threshold (d) are safeguards for other configurations; under the shipped parameters (c) never acts and (d) never acts on horizon rays. We accepted all four as the shipped algorithm; every statement in this paper about SE-MPPI is a statement about the controller with them.

(a) **Avoid lists.** The selection excludes rays that lead to the pose where entrapment was declared (1.0 m), to the origins of the last three hops, and to failed waypoints (0.6 m, at most six remembered). An avoided place counts only if it is ahead of the ray (less than 90° off), farther from the robot than its radius, not more than that radius beyond the end of the ray, and the ray passes within that radius of it. From the mouth of a pocket the ray back to the stall pose reaches the horizon and is closest to the goal bearing; without the exclusion the robot re-enters. Without the hop origins a rotated pocket made the selection swing between two places.

(b) **Movement watchdog.** A re-selection when the robot has moved less than 0.15 m in 3 s. On one development geometry MPPI stood still in front of a gap that a binary cost threshold called free but whose inflation cost was about 160; the advised 15 s period is too slow for Nav2's 10 s progress checker.

(c) **Waypoint floor.** A candidate waypoint closer than 0.2 m is not taken. Inert here: a selected ray has a free length of at least 0.6 m, so the waypoint is at least $0.6-0.234-0.05 = 0.316$ m away.

(d) **Clearance threshold.** The 0.6 m threshold on the free length of a ray applies to the horizon candidates and to the longest-ray fallback; the advice had no threshold on the fallback. On horizon rays it is inert here: the local costmap is 3 m wide and re-centres on the robot at its 5 Hz update, so a ray that reaches the horizon or the costmap edge is at least about 1.4 m long (shortest logged live: 1.42 m). On the fallback it excludes rays shorter than 0.6 m. The fallback never fired live: all 188 hops logged in campaign 2 (evidence trials included) took a horizon ray, so (d) did not act in any live trial. The mirror's switch for the additions removes the threshold from both places. Restoring it on the fallback in the advised arm changed no outcome on the U-trap family (29 of 50, no seed differs; `experiments/results_2d_v2_advised/FALLBACK_THRESHOLD.md`).

Measured ablation. We switched off (a) and (b) through their parameters (both avoid radii 0.0 m, stall time 10^6 s). The floor (c), the threshold (d) and the failed-waypoint list have no parameter, so the ablation leaves them on; (c) does not act under these parameters, and (d) acts only on the fallback. The test is the headless sweep of Sec. VI-C over the 56 valid development geometries, one run each. MPPI's noise cannot be seeded from the test, so each sweep is one process and the sweeps are independent. Table III gives the count per sweep. Without the additions the three archived confirmation sweeps pass 14 to 18 of 56, and an earlier, unarchived implementer sweep passed 19; with them 55 or 56 of 56. The additions were worked on until the development family passed, so the 56 of 56 is a development result and not a test result.

TABLE III

HEADLESS DEVELOPMENT SWEEPS: GEOMETRIES PASSED OF 56, ONE RUN EACH, PER SWEEP (PROCESS). "OFF" SWITCHES OFF ADDITIONS (A) AND (B) AS DESCRIBED IN THE TEXT. SOURCE: `EXPERIMENTS/CAMPAIGN2/PREREG.MD`. THE CONFIRMATION SWEEPS RAN AT COMMIT `B7E012A` (CONTROLLER SOURCES OF `507173A`); THEIR PER-GEOMETRY ROWS ARE IN `EXPERIMENTS/CAMPAIGN2/SWEEPS/`. RUN 3 WITHOUT THE ADDITIONS IS A RERUN (`OFF3B`): THE FIRST ATTEMPT STOPPED PRINTING ROWS AFTER 3 GEOMETRIES, IN THE TEARDOWN OF THE TEST RIG, AND WAS KILLED (`PREREG.MD`, LINES 225–234; ITS LOG IS KEPT OUTSIDE THE REPOSITORY). THE IMPLEMENTER AND REVIEWER SWEEPS ARE NOT ARCHIVED, THE COMMIT OF THE EARLIER IMPLEMENTER SWEEP WITHOUT THE ADDITIONS IS NOT RECORDED, AND NO SWEEP CAN BE SEEDED.

Sweep	Additions	Passed of 56
implementer	on	56
reviewer	on	55
confirmation, run 1	on	55
confirmation, run 2	on	56
confirmation, run 3	on	56
implementer, earlier	off	19
reviewer	off	16
confirmation, run 1	off	16
confirmation, run 2	off	18
confirmation, run 3	off	14

The two sweeps with 55 of 56 each had one geometry in which the robot never entered the pocket, so the run did not test a trap. With the additions the three confirmation sweeps passed 167 of 168 runs, and all 168 reached the goal; without them the three passed 48 of 168. No run had a contact or a `NoValidControl` throw. A geometry's outcome without the additions depends on its position in the sweep process: on a subset of four geometries, 3 processes per arm, the runs without the additions passed 8 of 12 when run alone and 4 of 12 in the sweeps, and with the additions 12 of 12 in both. Without the additions the primary geometry `w22_d13_r30_a10` passed 0 of 3 runs in the review. We therefore report counts per sweep and do not give a rate.

E. CBF safety filter (dynamic obstacles)

To make the barrier relative-degree one in (v, ω) for a unicycle, safety is enforced on a **look-ahead point** $P = (x, y) + L\,(\cos\theta, \sin\theta)$, which is fully actuated by (v, ω) through the Jacobian

$$G = \begin{bmatrix} \cos\theta & -L\sin\theta \\ \sin\theta & L\cos\theta \end{bmatrix}. \tag{1}$$

For each obstacle o with $d = P - p_o$ and effective radius $R = r + R_o + m$:

$$h_o = \|d\|^2 - R^2, \qquad \dot{h}_o = 2\,d^\top (G\,u - v_o). \tag{2}$$

The continuous-time CBF condition $\dot{h}_o + \alpha\, h_o \geq 0$ [37], [38], enforced once per control cycle, is linear in u. We solve a small QP (OSQP) [47] that minimizes

$$\|u - u_{\text{nom}}\|^2 + \rho\,\delta^2 \tag{3}$$

subject to the per-obstacle CBF rows (relaxed by a single slack $\delta \geq 0$ for feasibility) and the input box limits. Obstacles

are pruned to the nearest 20 (a compiled cap) by *clearance* (surface gap), not center distance. If the QP can only stay safe by relaxing the barrier ($\delta > \epsilon$) or fails, the controller sets the linear velocity to zero (forward or reverse; the controller's log calls it the forward velocity) while keeping the QP's turn (or, on solver failure, the box-clamped nominal turn) – stop rather than drive into an imminent collision. Static structure is intentionally **excluded** from the CBF (handled by the MPPI obstacle critic and costmap inflation); only obstacles that are moving and small enough to be a movable body are admitted (a track must pass the occupancy-persistence veto `is_dynamic`, move at least 0.1 m/s and have a radius of at most 1.0 m), which we found essential in a real costmap (Sec. VI-B).

Because this condition is checked once per control cycle (a sampled-data CBF), strict between-sample forward invariance would require a sampled-data margin tightening R by an $O(\dot{h}_{\max}\Delta t)$ term; in practice the obstacle-radius margin m absorbs this gap, and we report the empirical safety rate rather than claim continuous-time invariance between samples.

F. Escape–safety coordination: gain scheduling under a conditional invariance lemma

The coordinator resolves the gain:

$$\alpha = \begin{cases} \alpha_{\text{base}} & \text{not entrapped, or TTC} < \tau \text{ (imminent)} \\ \alpha_{\text{escape}} & \text{entrapped and TTC} \geq \tau \end{cases} \tag{4}$$

with $\alpha_{\text{escape}} > \alpha_{\text{base}} > 0$, where TTC is the minimum time-to-collision over tracked obstacles under the current command.[2] Raising α relaxes the constraint $\dot{h} \geq -\alpha h$ – permitting the robot to approach an obstacle faster, as an escape maneuver requires – while the TTC override snaps the gain back to α_{base} (safety first) when a dynamic obstacle is imminent. The next result states that, under explicit conditions, this schedule does not by itself give up forward invariance. It is a known property of CBFs with time-varying class-$\mathcal{K}$ terms and follows the same argument as the relative-degree-one case of the invariance result for adaptive CBFs with time-varying penalty functions [29, Thm. 3], which is stated there for Lipschitz-continuous controllers, whereas our switched gain yields a discontinuous command; related time-varying and optimized decay rates appear in [39] and [30]. We restate it with the standard integrating-factor proof for completeness and claim no new mathematics.

Proposition (invariance under a time-varying gain; after [29]). *Fix an obstacle j, suppose the safety-filter QP is feasible without slack throughout the interval considered (the shared $\delta(t) = 0$), and assume the continuous-time inequality $\dot{h}_j(t) \geq -\alpha(t)\, h_j(t)$ holds for almost every $t \geq 0$ – an assumption between samples, since discrete-instant QP feasibility does not by itself imply the intersample inequality (the sampled-data caveat of Sec. IV-E). Let $\alpha : [0, \infty) \to [\alpha_{\min}, \alpha_{\max}] \subset (0, \infty)$ be any measurable, positive, bounded gain schedule – in particular the coordinator's piecewise-constant switching between α_{base} and α_{escape} (Eq. 4) and its TTC override. If $h_j(0) \geq 0$, then $h_j(t) \geq 0$ for all $t \geq 0$ – the look-ahead point P does not enter obstacle j's inflated disc, as modeled (body safety approximate; see Sec. VII).*

Proof. Define the integrating factor $\varphi(t) = \exp\big(\int_0^t \alpha(s)\, ds\big)$, finite and strictly positive because $\alpha \leq \alpha_{\max}$. Then, almost everywhere,

$$\frac{d}{dt}\big[\varphi(t)\, h_j(t)\big] = \varphi(t)\big[\dot{h}_j(t) + \alpha(t)\, h_j(t)\big] \;\geq\; 0, \tag{5}$$

so $\varphi(t)\, h_j(t)$ is non-decreasing, hence $\varphi(t)\, h_j(t) \geq \varphi(0)\, h_j(0) = h_j(0) \geq 0$. Since $\varphi(t) > 0$, $h_j(t) \geq 0$ for all t. ∎

Positivity and boundedness are stronger than the argument needs: a nonnegative, locally integrable α already makes φ finite and positive. We keep the stronger hypothesis because it matches the schedule of Eq. (4).

Corollary. Under the slack-free and between-sample conditions of the Proposition, forward invariance of $\mathcal{C}_j = \{h_j \geq 0\}$ holds for *every* positive, bounded gain schedule – in particular for the coordinator's switching between α_{base} and α_{escape}, and for the TTC override that drops α back to α_{base}. If these conditions hold for every row, $\mathcal{C} = \bigcap_j \mathcal{C}_j$ is invariant; the argument is per-obstacle and gain-agnostic, so sharing a single $\alpha(t)$ across rows is immaterial to the invariance statement itself.

Under these conditions, raising α therefore does not permit $h_j < 0$; it only weakens the lower bound on $\dot{h}_j$, letting the robot approach $\partial\mathcal{C}_j$ faster (bolder detours) inside the same set. This is the design rationale for coordinating rather than stacking the two layers: in this idealized setting, the gain trades caution for maneuverability without giving up invariance. The guarantee is narrow. It concerns the look-ahead point, not the robot body; it covers only the tracked dynamic obstacles under the constant-velocity model, not static structure; it applies to an obstacle only while that obstacle stays in the tracked, unpruned set $\mathcal{O}$ (an obstacle that slows below the speed threshold for dynamic obstacles, merges into a cluster too large to be a movable body, or is pruned beyond the nearest few by clearance leaves the QP); it assumes the continuous-time inequality between samples; and it requires $\delta = 0$, so it does not cover any cycle in which the QP uses slack. Because the slack is shared, one imminent obstacle can momentarily loosen the margin on every other obstacle in the same QP solve, so we report slack usage empirically: in the 2D benchmark of Sec. VI-D the QP never engages slack on the mover-free families, and uses nonzero slack in 26–35 of 50 narrow-dynamic trials (max 0.47) and 47–48 of 50 dynamic-family trials (max 0.014) per CBF-carrying config (Sec. VI-D1). In cycles with $\delta > \epsilon$ the brake of Sec. IV-E sets the linear velocity of the QP command to zero, and bounded-prediction-error robustness is the subject of follow-up work (Sec. VII).

[2]The released controller's coordinator additionally holds α at α_{base} whenever the online conformal bound q on obstacle-prediction error exceeds a trust threshold (0.25 m by default; the calibrator is enabled by default), and that bound inflates the effective radius R of Sec. IV-E – the SE-Predict margin noted in Sec. II, not claimed by this paper. An implementation following Eq. (4) alone therefore reproduces the benchmark configurations of Sec. VI-D, not the released defaults.

G. Dynamic-obstacle tracking

The tracker clusters costmap cells at or above the inscribed-obstacle cost (253) and matches clusters to persistent tracks greedily, nearest track first, in cluster order; each track is matched to at most one current cluster within a gate, so two clusters cannot inherit one track's velocity, and a track survives up to 3 unmatched frames (`se_track_max_missed`). A split or merge can still shift the centroid of the matched cluster and so produce a phantom velocity (Sec. VI-E1). The tracker estimates a clamped constant velocity. The controller stamps each costmap frame with the node clock in the control cycle that first sees a changed grid, not with the costmap's update time; with the local costmap at 5 Hz and the controller at 10 Hz in the campaign, the time step of the velocity estimate is rounded to control cycles, and on a robot the costmap latency adds to this jitter. Its output $\{p_o, R_o, v_o\}$ feeds the coordinator (TTC) and the CBF filter. The constant-velocity model is a deliberate, replaceable baseline (Sec. VII).

Configuration that ran live. The campaign parameter file does not set `se_classify_static`, `se_predict_horizon` or `se_conformal`. Their code defaults are true, so every live SE trial ran with the occupancy-based static/dynamic classification (the `is_dynamic` veto above), the horizon prediction and the online conformal margin on. The 2D mirror has none of the three. The statements of this paper about the tracker and the CBF filter hold for this configuration only. A cell counts as static only after `se_static_min_frames` (10, the code default, not set in the campaign file) tracker frames of occupancy, so newly seen structure, such as a wall entering the rolling window, is a dynamic candidate during that period. We state this as a code fact; we did not establish that it caused the phantom admissions of Sec. VI-E1.

V. Implementation (Nav2-Native)

SE-MPPI is built on ROS 2 Jazzy + Nav2 [1]. The package `nav2_se_controller` ships four shared libraries and one executable: a plugin-free `se_mppi_core` (CBF filter, coordinators, clustering, static filter, predictor, tracker, path progress, opening search; the entrapment detector is header-only), the `escape_critic` MPPI critic (with the repulsion and the critic's gap search), the `safe_escape_controller`, and `mppi_rollout_check` with the `check_mppi_rollout` executable. Separating the plugin-free core from the two plugin libraries resolves a `class_loader` factory conflict that otherwise prevents the critic and controller from co-loading.

Two integration details proved essential and are easy to get wrong:

- **Critic namespace.** Nav2's `CriticManager` resolves a critics-list entry `NAME` to the plugin class `mppi::critics::NAME`. A critic in any other namespace can be unit-tested via pluginlib directly but will **never** load from the critics list. `EscapeCritic` therefore lives in `mppi::critics`.
- **Jazzy critic data uses xtensor**, not the Eigen of the `main` branch; the critic was written against the installed headers.

The QP uses `osqp-eigen` [47]. The build is reproducible via RoboStack (conda-forge) so the build does not depend on the host OS version (validated on Ubuntu 20.04 with an NVIDIA GPU). The package's modules for this paper (entrapment detector, CBF filter, coordinator, tracker, gap search, repulsion, path progress, and plugin loading) carry **44 unit tests** (gtest), plus linters; the wider package additionally carries tests for the follow-up papers' prediction stack (static/dynamic occupancy classification, trajectory prediction, conformal calibration) and multi-robot coordination module (SE-Predict, Multi-SE-MPPI), plus the live-injection ABI regression of Sec. VI-B, bringing the algorithm test suite (`test/test_*.cpp`) to 82 `TEST` cases across 13 files. The deployment tests in `test/deployment` (the controller on a lifecycle node inside the stock MPPI optimizer, the escape rules, the path-progress tracker, the rollout-direction guard and the headless closed-loop test) add 148 `TEST` cases in 10 files. The code evaluated in the headless sweeps and in the live campaign is the tag `campaign2-controller`; the mirror re-run and the design-stage benchmark of Appendix B ran at the commits named in Appendix A.

TABLE IV
What each result ran on. "Affected" means a binary that rolls the sampled trajectories out in the wrong direction for part of the heading range (Sec. VI-A); "correct" means `nav2_mppi_controller` 1.3.13 build _25 from RoboStack, which passes the rollout check.

Result	MPPI library
Mirror benchmark	none: the Python mirror imports nothing from ROS or Nav2
Headless sweeps	correct; the controller's configure-time rollout check is on by default and the rig does not override it
Campaign 1, 4 trials	affected (1.3.11 build _16)
Campaign 1, 44 trials	correct
Campaign 2, 88 trials	correct; the driver refuses to start unless the rollout check passes
July 2026 live runs	affected, according to the repository notes; not used for any claim here

VI. Experiments

Four kinds of evidence support this paper, and they ran on different things. Table IV lists which build of the MPPI library each used. The reason it matters is a defect in some distributed `nav2_mppi_controller` binaries (Sec. VI-A).

A. Live history: campaign 1 and the four defects

An earlier version of this paper reported live runs from July 2026 in which no run reached its goal, and attributed that to the host (a real-time factor of 0.3 to 0.6 and phantom costmap cells from transform smear). We withdraw that attribution. A pre-registered live re-run (campaign 1) drove the same U-pocket testbed in 48 registered trials at a real-time factor between 0.93 and 0.97: 4 on an affected build and 44 valid trials on the

correct build. None reached the goal: the 44 valid trials ended in 42 timeouts and 2 aborted goals, 13 of them with an escape episode. The raw files, the analysis and the root-cause note are in `experiments/results_pilot_rerun/`. The note names four defects of the controller and its parameter file, which act in this order.

1) **Forward-only speed range.** The pilot's `vx_min` was 0.1, so the robot could not stop or back off. In the 20 valid trials with that setting it crept into the back wall; the chassis touched the wall in 14 of them. Once all rollouts collide the optimizer throws `NoValidControl` every cycle.
2) **Detector skipped on a throw.** The first version updated its entrapment detector only after the optimizer call returned, so it never saw a stall that ended in a throw.
3) **Progress re-anchored on every replan.** A replan to the same goal re-anchored the furthest-reached path index, so one cell of motion (2.5 cm) along the new path counted as progress. All 70 closed escape episodes of the parameter set that allowed reversing ended at path index 1 or 2, at a median of 3.1 s.
4) **Escape did not route out.** In that parameter set, seven episodes of 24 to 83 s occurred in 5 of the 6 SE-MPPI trials, and the robot stayed within 0.34 m of its stopping point. The gap selection was not logged. From the pocket geometry and the observed heading we infer that it chose a sideways opening and rocked in place; this inference is not a measurement, and the repaired controller logs every selection.

None of the four exists in the Python 2D mirror of the first version, which used distance-to-goal progress, a clearance of 2.4 m on a 3.0 m window and a subgoal that replaced the goal. The benchmark result of the first version, kept in Appendix B, therefore described an idealized design and not the plugin.

A defective distributed binary. Some distributed builds of `nav2_mppi_controller` negate cosine and sine in the batch rollout integration for part of the heading range, so the optimizer scores its samples at mirrored positions; a controller built on them commands motion away from the goal for some headings. The upstream report (RoboStack/ros-jazzy#290) names the interaction of fast-math with xsimd's use of signed zeros as the suspected cause and was closed on 2026-10-10. The plugin carries a check, `check_mppi_rollout`, that compares 24,576 rollout points of the library it loads with the closed form at a tolerance of 1 mm, and `SafeEscapeController` refuses to configure on a library that fails it. The check guards only `SafeEscapeController`: any other controller that loads the same library is unguarded. Run `check_mppi_rollout` on every target install (the apt binaries are untested), and never set `se_enforce_rollout_check` to false on a moving robot. The July 2026 runs used an affected build; whether that decided their outcome is not established. We did not use any result obtained on an affected build in the claims below.

B. Deployment faults

Three integration faults surfaced in the live stack that the in-process unit tests did not catch, each reported as a deployment lesson:

- **Static walls in the CBF.** Feeding every inscribed-cost cluster (including static walls) into the CBF makes the barrier infeasible everywhere and freezes the robot. Scoping the CBF to clusters that the tracker classifies as dynamic (Sec. IV-E) stopped the wall freeze; it did not stop phantom admissions, which occurred in most live SE-MPPI trials (Sec. VI-E1).
- **Compute budget.** The per-cycle budget must be matched to the host (horizon preserved, rate and batch reduced). The comment in the campaign parameter file records what happened on this host otherwise: the 20 Hz, 56-step, batch-2000 configuration ran at about 5 Hz, stalled the controller server's executor past the 4 s lifecycle bond, and the lifecycle manager tore the whole stack down. With the stock bringup the bond timeout cannot be set from the parameter file. The released defaults (20 Hz, 56 steps, batch 1000) were not evaluated. Deployment warning: measure the achieved controller rate on the target (Nav2 warns when the controller misses its desired rate) and expect a lifecycle teardown when the executor starves.
- **xtensor ABI mismatch.** The installed `libmppi_controller.so` is compiled with `XTENSOR_USE_XSIMD` + AVX2 (an `xsimd::aligned_allocator` element type), whereas our plugin compiled xtensor with the default `std::allocator`. Because `CriticData`'s tensors cross the pluginlib boundary by reference, the first `EscapeCritic` write to `data.costs` was an ODR violation that segfaulted the entire Nav2 process on the first live escape injection (3/3 reproducible). The in-process unit tests never caught it, because they compile every translation unit with one consistent flag set. Fixed by compiling the critic against the installed headers' flags (per-target, since a global AVX2 flip breaks the separately built OSQP–Eigen boundary); a regression test replays the injection path with live tensor shapes (`test_escape_injection_live_shapes.cpp`), and `scripts/check_simd_abi.py` checks the storage type in CI.

C. Headless closed-loop development test

Before any further live trial we built a headless closed-loop test that would have caught defects 2 to 4. It runs the real optimizer (batch 600, 28 steps, $dt = 0.1$) and the controller on a lifecycle node with a drawn costmap, integrates the commands on a unicycle at the control period, replans a straight path from the robot every 1.0 s as the behavior tree does, and emulates the controller server by catching `NoValidControl` and applying zero velocity. It is the development loop for the escape.

Robot radius in the rig. The controller in the rig took its robot radius, 0.114 m (logged as `robot_radius=0.114` in every controller start line of the confirmation sweep logs, which lie outside the repository; per-log counts in `experiments/campaign2/sweeps/`

robot_radius_lines.txt), from the default footprint of Costmap2DROS, which the rig does not set. The rig's drawn costmap and its contact model use 0.22 m; the live controller used 0.234 m (Sec. IV-C). The radius enters the waypoint distance and the CBF disc, so for the same ray the waypoint of the rig lies 0.12 m farther out than the live one. We did not change the rig.

Family rule. The test pockets are generated from a written rule, fixed in a committed file before the first run (test/deployment/UTRAP_DEV_GEOMETRIES.md). A pocket has width $w \in \{1.3, 1.6, 2.2, 2.6\}$ m, depth $d \in \{1.1, 1.5, 1.8\}$ m, rotation $\theta \in \{-25, 0, 15, 30\}^\circ$ and symmetric or asymmetric side walls (top wall at 0.6 of the bottom wall): $4 \times 3 \times 4 \times 2 = 96$ combinations. A combination is valid only if the straight start–goal path enters the mouth, start and goal are at least 0.8 m from every wall, the straight line is blocked, and the pocket is not the live development scenario. The generator finds 56 valid geometries. Two primary geometries, outside the grid, gate the test (w22_d13_r30_a10 and w16_d15_r-15_a6). A run passes if the robot enters the pocket, leaves it within 60 s, reaches within 0.5 m of the goal within 150 s, and has no wall contact and no NoValidControl throw. The development scenario s1_utrap is excluded from the family, and a guard test fails if the name of a held-out world appears in the repository outside the campaign directory.

Result and its status. The shipped controller passes 55 or 56 of the 56 valid geometries per sweep, and without additions (a) and (b) 14 to 18 in the three archived confirmation sweeps (19 in an earlier, unarchived sweep; Table III). The additions were worked on until this family passed, so the family is a development set, not a test set. In addition to the family, the additions were developed on static gap tests at the stall pose of the live development scenario s1_utrap; that is a use of a live scenario in the design of a parameter, and the reason s1_utrap is a development scenario and not a test.

The twin of s3. The held-out campaign pocket s3_ctrap was specified before the tuning and authored by a separately prompted coding assistant from the geometry alone; it was not driven live and not run headless. It is not outside the family: reflected about $y = 0$ it matches the family member w22_d15_r-25_a6 to within 0.1 m in width, 0.05 m in depth and 0.09 in the wall ratio. The additions were worked on until that twin passed in the sweeps; run alone with the additions off, the twin also passed 3 of 3 (PREREG.md), so it is not evidence that the additions were needed for it. We therefore do not use s3 as the test of transfer; Sec. VI-E uses a second held-out pocket that lies outside the family.

D. Randomized benchmark on a Python 2D mirror

Every quantitative result in this section comes from a Python 2D mirror, not from the Nav2 controller. The mirror re-implements the controller's primitives in NumPy and OSQP [47]: the MPPI sampler, a repulsion of the same form with other parameters (Appendix), the ray cast, the look-ahead-point CBF-QP, the α coordination with TTC override, and, in the version used here, the shipped escape: progress on an anchor path with simulated one-second straight replans, the horizon-ray selection with hysteresis, the frozen-waypoint hops, the one-cycle lag, and a zero-command mode when every rollout collides. The mirror keeps running after such a cycle, as a controller server with a failure tolerance of -1 would, not as the campaign's tolerance of 0; in the shipped arm one F trial on dynamic (a success) and one on narrowdyn (a collision) passed through such cycles, and no utrap trial did. A flag switches the four additions of Sec. IV-D on (the shipped algorithm, variant v2) or off (the design as advised, variant v2_advised). A test calls the C++ selection, ray cast, progress chain and detector through a shim and compares them with the mirror; the hop state machine and the one-cycle lag were checked by reading only. The mirror still differs from the controller in the ways the Appendix lists: it computes geometry analytically against circular obstacles, gives the CBF and the TTC the true obstacle states, and uses its own cost. It does not run Nav2's progress checker or behavior tree, so it is a controller-level benchmark. We re-ran the full benchmark, with the same families, seeds, configs and budget as the design-stage benchmark of Appendix B, on this mirror, so that the paper's numbers describe the algorithm that ships.

Setup. Four seed-deterministic scenario families exercise the two weaknesses and their conjunction: utrap (a finite U/C pocket with the goal behind the back wall – a static local minimum), clutter (a random field of 5–9 circular obstacles with a guaranteed footprint-clear path), dynamic (1–2 pedestrian-like crossing movers timed to reach the robot's path), and narrowdyn (the utrap pocket *plus* movers sweeping the pocket mouth during the escape window). Six configs toggle the mechanisms: **A** stock (no escape/CBF), **C** escape only, **D** CBF only, **E** escape+CBF with $\alpha_{\text{escape}} = \alpha_{\text{base}}$ (independent), **F** full SE-MPPI ($\alpha : 2 \to 6$ + TTC override, coordinated), and $\mathbf{F}^-$ (F with the gap search off). C and F^- exist only in the mirror: the Nav2 overlays of the same names in experiments/configs/ablations.yaml (C_escape_detect, F_minus_gap) do not reproduce them under the shipped controller, which clears the shared entrapment flag while se_enabled is false and keeps its own opening search when the critic's gap search is off; no Nav2 result for them is reported. Config A is stock MPPI without Nav2's progress checker and behavior-tree recoveries [3], [4], so it is a controller-level baseline, not the full Nav2 stack. Each family is a pure function of an integer seed, so every config is evaluated on an *identical* scenario set, as the paired McNemar design requires: 4 families $\times$ 50 seeds $\times$ 6 configs $=$ 1,200 paired trials per arm, 40 s sim budget each; these are 200 seeded worlds, each run under all six configs, so the trials are paired rather than independent. Success carries a Wilson 95% CI [48]; the primary test is McNemar's [49] on paired outcomes; continuous metrics use the Mann–Whitney U test [50] with Cliff's δ [51], an unpaired comparison that ignores the pairing, with time-to-goal and path length taken over successful trials only, so we read these as descriptive; all p-values are Holm–Bonferroni-corrected [52] within each family. The two arms (shipped v2, advised v2_advised) are 1,200 trials each, run at the commit named in the Appendix, with 6 workers.

TABLE V
RANDOMIZED BENCHMARK ON THE PYTHON 2D MIRROR OF THE SHIPPED ALGORITHM (NOT THE NAV2 CONTROLLER): SUCCESS RATE IN % (WILSON 95% CI), $N = 50$ PER CELL, SHIPPED ARM `v2`. CONFIGS A, D AND F$^-$ DO NOT USE THE OPENING SELECTION OR THE HOPS (F$^-$ KEEPS DETECTION AND REPULSION), SO THEY ARE THE SAME AS IN THE ADVISED ARM.

Config	U-trap	Clutter	Dynamic	Narrow-dyn
A stock	0 [0,7]	52 [39,65]	80 [67,89]	0 [0,7]
C escape	80 [67,89]	72 [58,83]	80 [67,89]	72 [58,83]
D CBF	0 [0,7]	52 [39,65]	82 [69,90]	0 [0,7]
E indep.	80 [67,89]	72 [58,83]	84 [71,92]	60 [46,72]
F SE-MPPI	80 [67,89]	72 [58,83]	84 [71,92]	60 [46,72]
F$^-$ no-gap	0 [0,7]	52 [39,65]	84 [71,92]	0 [0,7]

TABLE VI
F (SE-MPPI) SUCCESS IN % (WILSON 95% CI) IN THE TWO ARMS OF THE RE-RUN: SHIPPED ALGORITHM (`v2`) AND THE DESIGN AS ADVISED, WITHOUT THE FOUR ADDITIONS (`v2_ADVISED`); $N = 50$ PER CELL.

Arm	U-trap	Clutter	Dynamic	Narrow-dyn
`v2` (shipped)	80 [67,89]	72 [58,83]	84 [71,92]	60 [46,72]
`v2_advised`	58 [44,71]	62 [48,74]	84 [71,92]	44 [31,58]

The escape layer is the measured effect on the trap families. On both trap families the effect survives Holm correction (Table VII; per-config success rates in Table V): success rises from 0% (stock A, CBF-only D, and no-gap F$^-$ all time out in the pocket) to 80% on `utrap` (40/50 discordant pairs, $p_{\text{adj}} = 1.1 \times 10^{-8}$) and to 60–72% on `narrowdyn` (30/50 discordant for F, $p_{\text{adj}} = 1.9 \times 10^{-6}$). **F$^-$ scores** 0% on both trap families, as expected by construction: with the opening search off, the escape has no path to follow. Table VI gives the headline of this section: with the four additions the U-trap success is 80% (40 of 50), and without them 58% (29 of 50). On `narrowdyn` the arms give 60% and 44%. Without the additions the escape still lifts U-trap success from 0% to 58%; the additions raise it to 80%. Paired by seed, 11 U-trap seeds succeed only with the additions and none only without them; on `narrowdyn` the split is 10 and 2. We ran no test between the arms, so this comparison is descriptive. The U-trap number is lower than that of the design-stage benchmark of Appendix B, and we report the lower number. On `clutter` the shipped escape raises success from 52% to 72% (0/10 discordant pairs, Holm-adjusted $p = 0.031$), at the cost of about 4.8 s longer average time-to-goal over successful trials; in the advised arm the gain is smaller ($52\% \rightarrow 62\%$, not significant after Holm). On the pure-`dynamic` family, which contains no trap, the escape is neutral (stock→F: 0/2, $p = 1$). Fig. 2 gives the full per-family picture.

1) Escape–safety coordination is null in this regime: The load-bearing contrast for the C3 contribution is **E (independent) vs. F (coordinated)**: identical mechanisms differing only in whether the CBF gain is coordinated with the escape phase. *Across all 200 paired scenarios this contrast is null* (Table VII, Fig. 3): the discordant counts are $0/0$ in every family, McNemar $p = 1$, and every continuous effect size is $|\delta| \leq 0.002$. The same holds in the advised arm ($0/0$ in every family, $|\delta| \leq 0.005$). The gain does engage as designed ($\alpha_{\max} = 6$ during escape, TTC override active). The committed per-trial slack telemetry (`slack_max` in `trials.csv`; per-config rates in `stats.json` and `tables.md`) locates where the barrier actually works. On the mover-free families (`utrap`, `clutter`) the dynamic-obstacle CBF sees no obstacles at all: slack is zero in every trial, E and F issue identical commands, and the null there is structural. On `narrowdyn` the barrier *does* engage and even relaxes – the QP uses nonzero slack in $35/50$ trials for E and F ($26/50$ for D and F$^-$; max slack 0.47) – and on `dynamic` in 47–$48/50$ trials (max 0.014), yet the E-vs-F contrast stays null. The correct reading is that **the outcomes are insensitive to the α schedule**: where the barrier is quiescent the two configs coincide exactly, and where it is active the base and escape gains admit escape maneuvers whose outcomes are indistinguishable at $N = 50$. We report this honestly: **in this 2D benchmark the coordination produces no measurable outcome difference over independent escape+CBF.** The result is consistent with, and delimits, the Proposition of Sec. IV-F: coordination is a *mechanism with a conditional invariance property*, not, in these scenarios, an outcome improvement. An outcome-level benefit would require a regime where the choice of α changes which escape maneuvers are admitted at outcome-deciding moments – faster or persistent movers, tighter margins, a dynamic gap the escape route must thread while the barrier is live; this generator's pedestrian-like movers produce barrier engagement but not that sensitivity. The live campaign cannot test it either, because its pockets contain no dynamic obstacle. Constructing and measuring that regime is future work (Sec. VII); we claim no coordination benefit we have not measured.

TABLE VII
KEY PAIRED CONTRASTS (MCNEMAR, HOLM-ADJUSTED; b/c DISCORDANT PAIRS, b = BASELINE WINS, c = F WINS), PYTHON 2D MIRROR, SHIPPED ARM. THE ESCAPE CONTRAST IS DECISIVE ON THE TRAP FAMILIES; THE COORDINATION CONTRAST (E→F) IS NULL EVERYWHERE.

	Escape (stock→F)		Coord. (E→F)	
Family	b/c	p_{adj}	b/c	p_{adj}
U-trap	0/40	1.1×10^{-8} *	0/0	1
Clutter	0/10	0.031 *	0/0	1
Dynamic	0/2	1	0/0	1
Narrow-dyn	0/30	1.9×10^{-6} *	0/0	1

Two CBF observations bear on Sec. VII. Against body-grazing crossers (`dynamic`) the look-ahead-point filter leaves the collision rate essentially unchanged ($20\% \rightarrow 16\%$; descriptive – the committed analysis carries no paired collision-outcome test). And in the cramped `narrowdyn` pocket the CBF configs show a *higher* collision rate than their no-CBF counterparts ($18\% \rightarrow 32\%$, descriptive) and cost 12 pp success ($72\% \rightarrow 60\%$; success McNemar F vs. C: $b/c = 6/0$, raw $p = 0.031$, Holm-adj. 0.438 – directionally consistent though not significant after correction at $N = 50$).

E. Pre-registered live campaign 2

Question. Does the repaired controller escape a pocket live, in a full ROS 2 Jazzy + Nav2 + Gazebo stack? The pre-

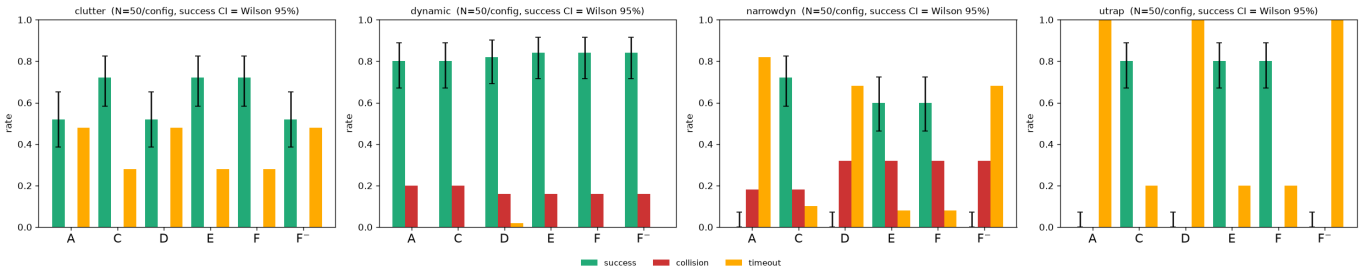


Fig. 2. Randomized benchmark on the Python 2D mirror of the shipped algorithm: success / collision / timeout per family across configs A–F$^-$ (Wilson 95% error bars on success, $N = 50$). On the trap families (`utrap`, `narrowdyn`) the escape configs (C = escape-only, E = independent escape+CBF, F = coordinated) separate from stock A, CBF-only D, and no-gap F$^-$, which time out at 0% success.

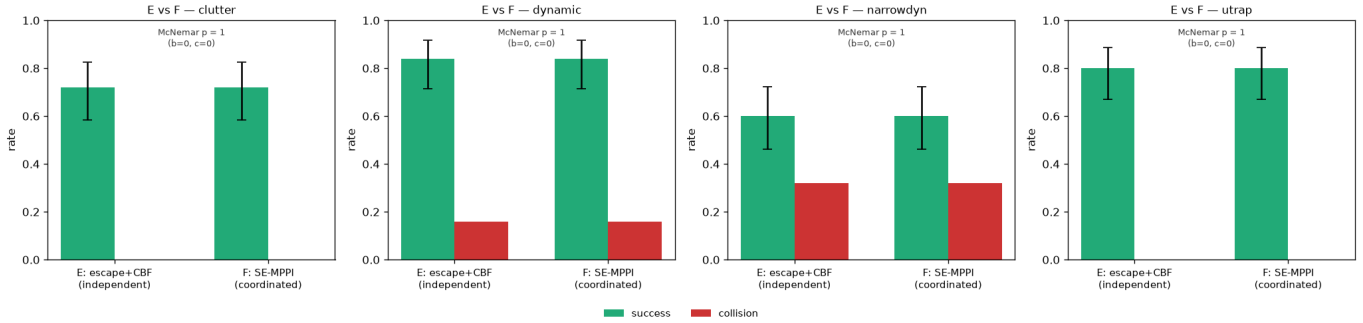


Fig. 3. The coordination contrast E vs. F per family in the Python 2D mirror of the shipped algorithm, annotated with the McNemar p-value. The independent (E) and coordinated (F) stacks are statistically indistinguishable in every family – the null result of Sec. VI-D1.

registration (`experiments/campaign2/PREREG.md`) was frozen on 2026-10-18, before any counted trial, with the controller, the parameter file, the worlds, the plan and the analysis code listed with their sha256 in a manifest and marked by the local tag `campaign2-controller`. Every trial records the commit, the build stamp, the hashes and the installed libraries; the driver refuses a trial whose installed files differ from the frozen ones.

Design. The testbed is a walled arena. The start is $(-3, 0)$ and the goal $(3, 0)$ in every scenario. The localization map is an empty box, so the pocket is absent from the planner's map: the global planner draws a straight path into the pocket, and only the controller can get the robot out. Four scenarios:

- `s1_utrap`: a pocket 1.7 m wide and 1.85 m deep; the development scenario.
- `s3_ctrap`: a three-wall pocket with an oblique back wall, 2.1 m wide and 1.45 m deep, rotated 25°. Held out in a limited sense: not driven live, not run headless, authored by a separately prompted coding assistant that saw only the geometric specification, but it has a near mirror twin in the development family (Sec. VI-C). Reported, not decision-bearing.
- `s4_ltrap`: an L-shaped dead end (Fig. 4). The author specified it before anything was run on it; a separately prompted coding assistant that saw only the geometric specification, and not the controller, the family rule, the tuning or the campaign-1 analysis, wrote the world. The specifier (the author) knew the design, so only the world file was authored blind. Its corridor is 2.05 m deep against 1.85 m for `s1_utrap`, the deepest development pocket: 0.20 m, 11 percent deeper, and it has a closed bent leg, a shape the family does not contain. It lies inside the family's span in width (1.3 m) and rotation (−15°). It also fails the family's rule V1 (the straight line enters the corridor with a margin of 0.10 m, not 0.3 m). It carries the claim of transfer.
- `s0_open`: the testbed without the pocket; a control for false escapes and for the health of the stack.

Four configurations cross the controller with the behavior tree: C1, stock MPPI with Nav2's default recovery tree; C2, SE-MPPI with that tree; C3, stock MPPI with a replanning-only tree in which the first `FollowPath` failure ends the goal and the progress-checker allowance is 30 s; C4, SE-MPPI with that tree. Stock MPPI in C1 and C3 is the `SafeEscapeController` plugin with `se_enabled` false and without `EscapeCritic` in the critics list: it passes the MPPI command through unchanged and still enforces the rollout check; it is not a separate stock plugin. The parameter file is the campaign-1 file with reversing allowed (`vx_min` −0.35), Nav2's default `failure_tolerance` of 0 and progress checker (0.5 m in 10 s), and the MPPI block unchanged (10 Hz, 28 steps, $dt = 0.1$, batch 1500, inflation 0.45 m). C1 and C2 run on s0, s1 and s3; C3 and C4 on all four; six repetitions per cell. Trials ran in a pre-registered fixed order, not a random one: the cell order rotates with the repetition (`plan.py`; one place per repetition in Phase A, two in Phase B). In Phase A the order on s1 was C4, C3; C3, C4; C4, C3 for repetitions 0 to 2, then C4 alone for repetitions 3 to 5. The plan has 88 trials: Phase A (the gate: C4 on s1 six times and C3 on s1 three times, 9 trials) and Phase B (79

trials, which include 4 uncounted evidence trials and the 12 trials on s4). The driver refuses Phase B and every trial on s3 and s4 until Phase A is complete and its gate has not fired. If C4 on s1 reaches the goal in 1 of 6 or fewer, the campaign stops. A trial starts only when the one-minute load average is below 2.0 at two checks 30 s apart and the busy CPU over those 30 s is below 0.5 core, and it is valid if the foreign CPU averages at most 1.0 core and stays at most 3.0 cores in every 30 s window.

Host and tolerances. Every trial ran on one workstation, under WSL2, with an AMD Ryzen 5 5500GT (12 threads) and simulated time (`use_sim_time`). The control loop was paced by the wall clock, not by simulated time: over the 46 SE-MPPI trials the controller ran 9.83 to 10.00 cycles per wall-clock second and 10.14 to 10.42 per simulated second (post hoc, `CBF_ADMISSIONS.md`). The timers that count cycles (the stall window, the watchdog and the re-selection period) were therefore wall-clock durations, a few percent shorter in simulated time. On a robot the two clocks coincide. Over all 88 trials the stack used 3.2 to 4.2 cores on average per trial, the peak one-minute load average was 1.79 to 8.35, and the lowest real-time factor over a 10 s window was 0.882 (0.944 in the SE-MPPI pocket trials); only the 12 C1 pocket trials logged missed control cycles (2 to 5 each). The parameter file loosens tolerances for the lag of the simulated transform tree: `transform_tolerance` 0.5 s on the controller, the costmaps, the behavior server and the collision monitor (1.0 s for AMCL). It sets `source_timeout` of the collision monitor to 5.0 s for a different reason, which its comment records: on this host the scan lagged simulated time by more than 1 s under load, and at 1.0 s the monitor declared the source invalid and set `cmd_vel` to zero in the middle of an escape. Campaign 2 did not measure scan age, which is also the input of the costmaps and the tracker. The collision monitor stayed in the command chain (`cmd_vel_smoothed` to `cmd_vel`); its log in the 88 trials holds no line above INFO level (`CBF_ADMISSIONS.md`). Do not copy these values to a robot, and measure the sensor latency on the target. The only `collision_monitor` polygon is disabled (`enabled: False`), so the live results had no active `collision_monitor` zone. It was disabled in the earlier smoke tests because its approach polygon (`time_before_collision` 1.2 s) slowed the robot almost to a stop in the narrow testbed rooms (`RUN.md`). No escape, live or headless, ran with an active approach polygon. A deployer who enables it, as we advise, must test escapes again, because the monitor may slow or stop the turns in a pocket. The MPPI cost critic ran with `consider_footprint` false, a point robot behind an inflation radius of 0.45 m, at least the 0.22 m radius of the circular simulated robot; a robot with a non-circular footprint needs `consider_footprint` true and its own validation.

Success criterion. A trial *reached* the goal if the Gazebo ground-truth position of the robot is within 0.5 m of the goal at the end of the drive and the chassis never touched a wall (distance at least 0.01 m in every sample). Nav2's own `SUCCEEDED` is recorded and not used.

Analysis and decision rule. Counts per cell with Wilson

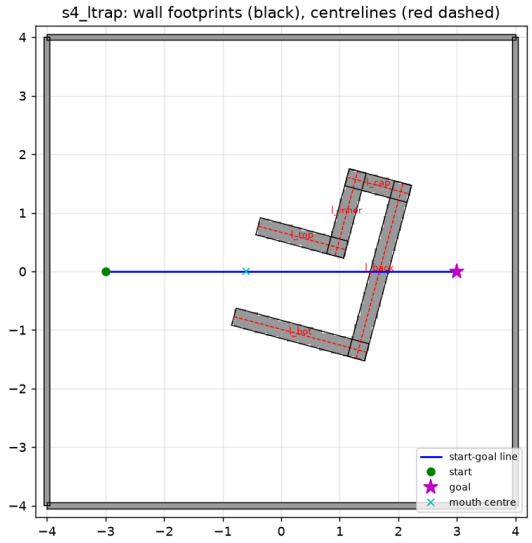


Fig. 4. The second held-out pocket `s4_ltrap` (wall footprints in black, centre lines dashed red), the straight start–goal line, and the mouth centre. Plot written by the coding assistant that wrote the world, `experiments/campaign2/s4_ltrap_geometry.png`.

95% intervals; two-sided Fisher exact tests [53] for C4 against C3 (s1, s3, s4 separately), C2 against C1 (s1, s3), and C4 on s3 and on s4 each against C4 on s1 (transfer): seven contrasts at a level of 0.05, no correction, no pooling over scenarios. Only two of them decide the branch, each once. The hypothesis on a pocket is *yes* if and only if the Fisher p of C4 against C3 on it is below 0.05 and C4 reached the goal more often. The branch *demonstrated* requires yes on s1 and yes on s4; s3 does not change the branch; no on s1 is *not demonstrated* whatever s4 shows. Requiring both s1 and s4 to be *yes* is an intersection-union test, so the probability of the branch *demonstrated* is at most 0.05 without a correction whenever at least one of the two pockets has no effect. Because s1 is a development scenario, its *yes* is not confirmatory, and the evidence of transfer is the s4 contrast alone ($p = 0.0152$). With six trials a cell, C4 at 5 of 6 against C3 at 0 of 6 gives $p = 0.0152$ and 4 of 6 gives $p = 0.0606$. The comparison of C2 with C1, the control, the transfer contrasts and all time comparisons are descriptive. The wording for each branch was copied before the trials from the design advice into the pre-registration.

Amendments and deviations. Every amendment is dated in the pre-registration. We list them all, because one was written after counted trials had run.

1) *Amendment 1 (2026-10-18, before any trial).* The author accepted additions (a) to (d) as the shipped algorithm and added `s4_ltrap`, with C3 and C4 on it, because `s3_ctrap` has a twin in the development family; the plan grew from 76 to 88 trials and the decision rule moved the claim of transfer to s4.
2) *Driver import failure (2026-10-22, before any trial).* The first start of Phase A ended at once with an `AttributeError`: the driver imported the analysis script of campaign 1 instead of its own. No simulator or trial ran. *Amendment 2* fixed the import by file path and added a regression test; the analysis rules, plan, worlds,

controller and thresholds did not change.

3) *Provenance refusal (2026-10-22, after the first counted trial).* After the first trial (C4 on s1, repetition 0) the provenance check refused the next one, because Python had written a bytecode cache into the installed, frozen launch directory. The author deleted the cache and made the directory read-only. No frozen file, code, parameter or environment variable changed, and the trial stayed counted, because it ran on a verified install.
4) *Amendment 3 (2026-10-22), post hoc.* The stopping rule for a controller crash fired after the counted trial `s4_ltrap` C3 repetition 0 (Sec. VI-E1). The amendment was written after the rule had fired and changes how the rule reads the evidence: a launched process that exits with a non-teardown code is a *teardown crash*, reported and not stopping the campaign, only if the process itself logged the termination signal inside the teardown window and before the launch logged its death; every other death stays a controller crash. Without the amendment the registered rule would have stopped the campaign at that point, and the remaining trials would not have run under it. It also departs from the rule "no source change after trial 1": the analyzer, the driver's two teardown time stamps, the provenance module and the manifest module changed; the controller, plan, worlds and parameters did not. The tag moved with it, so trials recorded before it carry the earlier freeze identity, which the analysis accepts for exactly that earlier set. It was written after 16 counted trials, all at the tag commit `012f4f1`. At that point the author knew these outcomes: on s1, C4 complete at 5 of 6, C3 failed in repetitions 0 to 2, C1 0 of 1 and C2 0 of 1; on s3, C1 0 of 1, C2 1 of 1, C3 0 of 1 and C4 1 of 1; on s4, C3 0 of 1 (the crash trial). No C4 trial on s4 had run. All six C4 trials on s4 and C3 repetitions 1 to 5 on s4 ran after the amendment, so the s4 contrast exists only under the amended rule. The registered rule required stopping the campaign or restarting it as attempt 2; we did neither. From `012f4f1` to `6a50723`, the diff of `analyze.py` touches the classification of crashes, the identity check that accepts the earlier trials, one recorded start time and the report sections; it does not touch the outcome logic (reached, contact, cells, contrasts, decision).

1) Results: All 88 trials ran and were valid; no attempt was invalidated by host load. The whole-drive real-time factor was between 0.93 and 0.97. Table VIII gives the counts, and Table IX the contrasts. No trial had a wall contact. SE-MPPI logged an escape in every pocket trial of C2 and C4 before the chassis left the pocket polygon, and no pocket trial reached the goal without an escape; the SE trials had no optimizer failure, no missed control cycle and invoked no recovery.

The pre-registered rule selects the branch *demonstrated*: C4 against C3 is yes on s1 and yes on s4. The pre-registered wording for that branch, copied from the design advice and quoted here verbatim, is:

> "SE-MPPI without recoveries reached the goal in k of 6 on s1 and k' of 6 on the held-out s3; stock without recoveries in 0 of 6; with Nav2's default recovery tree, stock m of 6 and SE-MPPI m' of 6", Fisher exact p per contrast, counts and Wilson intervals only, "on one simulated host". s1 is the development scenario; s3 carries the claim. Section 4.4 and the abstract's "no live run has yet reached its goal" are replaced; the pilot's host attribution is withdrawn (campaign 1: RTF 0.96).

TABLE VIII
CAMPAIGN 2: TRIALS THAT REACHED THE GOAL OUT OF 6 (WILSON 95% INTERVAL), BY SCENARIO AND CONFIGURATION. C1: STOCK WITH NAV2'S DEFAULT RECOVERY TREE; C2: SE-MPPI WITH IT; C3: STOCK WITH REPLANNING ONLY; C4: SE-MPPI WITH REPLANNING ONLY. – = NOT RUN.

Scenario	C1	C2	C3	C4
s0_open	6 [0.61,1.00]	6 [0.61,1.00]	6 [0.61,1.00]	6 [0.61,1.00]
s1_utrap	0 [0.00,0.39]	3 [0.19,0.81]	0 [0.00,0.39]	5 [0.44,0.97]
s3_ctrap	0 [0.00,0.39]	2 [0.10,0.70]	0 [0.00,0.39]	4 [0.30,0.90]
s4_ltrap	–	–	0 [0.00,0.39]	5 [0.44,0.97]

TABLE IX
CAMPAIGN 2: TWO-SIDED FISHER EXACT p OF THE REGISTERED CONTRASTS (REACHED OUT OF 6 IN EACH ARM). THE FIRST AND FIFTH ROWS DECIDE THE BRANCH; THE OTHERS ARE REPORTED AND DECIDE NOTHING.

Contrast	A	B	Fisher p
C4 vs C3 on s1 (development)	5	0	0.0152
C4 vs C3 on s3 (twin in family)	4	0	0.0606
C2 vs C1 on s1	3	0	0.1818
C2 vs C1 on s3	2	0	0.4545
C4 vs C3 on s4 (outside family)	5	0	0.0152
C4 on s3 vs C4 on s1 (transfer)	4	5	1.0000
C4 on s4 vs C4 on s1 (transfer)	5	5	1.0000

The pre-registration says to read "the held-out s3" and "s3 carries the claim" as `s4_ltrap`, and k' as the count of C4 on s4 (amendment 1). Filled with the counts of Table VIII: SE-MPPI without recoveries reached the goal in 5 of 6 on s1 and 5 of 6 on the held-out s4; stock without recoveries in 0 of 6; with Nav2's default recovery tree, stock 0 of 6 and SE-MPPI 3 of 6, on one simulated host (the two counts under the default tree are on s1). The Fisher exact p values are in Table IX, and the Wilson intervals in Table VIII. s1 is the development scenario; s4 carries the claim. The earlier statement that no live run has reached its goal is replaced, and the host attribution of the July runs is withdrawn (campaign 1: real-time factor between 0.93 and 0.97).

Reading, and what it does not show. The two contrasts that decide the branch have $p = 0.0152$ each, with five of six against zero of six. A single failure more in the SE cell would have given $p = 0.0606$ and no claim, so the result is as narrow as six repetitions allow, and the Wilson interval of 5 of 6 is $[0.44, 0.97]$. Three results are not significant and are not claims of effect: C4 against C3 on s3 (4 of 6 against 0 of 6, $p = 0.0606$; the pre-registered rule marks this "no", and the pre-registered sentence requires that both be reported); C2 against C1 on s1 (3 of 6 against 0 of 6, $p = 0.1818$) and on s3 (2 of 6 against 0 of 6, $p = 0.4545$). In particular the live data do not show that SE-MPPI helps inside Nav2's default recovery tree beyond what six trials can resolve: the direction is favourable and the intervals overlap. **Forced stops without a moving body.** No pocket held a moving body, yet the brake

fallback of Sec. IV-E (linear velocity, forward or reverse, set to zero for one control cycle of about 0.1 s, so in a pocket it also stops a reverse out in that cycle) fired in the SE pocket trials of s3 and s4. The QP failure counter is 0 in all 88 trials, so every forced stop was a brake on barrier slack, not on a solver failure. The counts come from the last diagnostics message of each trial, which the controller publishes once per wall-clock second, so stops in the final second are missing and the counts are lower bounds. The count of forced-stop cycles at the last diagnostics message of each trial was, per repetition: s3 C2 $5, 9, 9, 7, 8, 16$; s3 C4 $8, 8, 7, 5, 12, 8$; s4 C4 $1, 1, 0, 6, 0, 2$; and zero in every s1 trial and on `s0_open`. The filter returns the nominal command unchanged when the tracker admits no obstacle, so a forced stop means that the tracker admitted phantom obstacles: costmap clusters that passed the `is_dynamic` veto, the speed threshold and the radius limit of 1.0 m. Admissions were not limited to pockets. The diagnostics field `cbf_obstacles`, the number of obstacles passed to the CBF in the cycle of the message (sampled once per wall-clock second, so the counts are lower bounds), was above zero in at least one sample in 42 of 46 SE-MPPI trials (evidence trials included), among them 8 of 12 on `s0_open`, with at most 2 obstacles in a sample; forced stops occurred only on s3 and s4 (post hoc, `experiments/campaign2/CBF_ADMISSIONS.md`, from the raw archive). An admitted obstacle that needs no slack can still change the QP's (v, ω) without a stop; no diagnostic records whether it did, so we do not know how often the CBF altered commands. This was not a designed test of the coordination, and we did not test whether the stops or the admissions changed any outcome. Deployment warning: a robot can be slowed, turned or stopped by obstacles that do not exist, in open space as well as in a pocket; read both the forced-stop counter and `cbf_obstacles` of the diagnostics on the target map. On the trap-free control `s0_open` all 24 trials reached the goal and SE-MPPI logged no escape. The transfer contrasts are descriptive: C4 reached the goal in the same number of trials on s4 as on s1 (5 of 6), but its median wall-clock time to goal over reached trials (the registered measure) was 48.3 s on s4 against 34.3 s on s1 (exact Mann–Whitney $p = 0.0079$, descriptive); in simulated time the medians are 46.7 s and 33.1 s.

Trials that Nav2 called successful but did not reach the goal. Of the 88 trials, 13 are counted `NOT_REACHED_ABORTED` although Nav2 reported `SUCCEEDED`: 11 registered trials (3 of C2 and 1 of C4 on s1, 4 of C2 and 2 of C4 on s3, 1 of C4 on s4) and the 2 evidence trials. In every one the robot's Gazebo ground-truth distance to the goal at the end of the drive was above 0.5 m, between 0.54 and 1.66 m. The two definitions differ. The goal checker of Nav2 has a tolerance of 0.25 m and works from the localization pose, an AMCL estimate on a localization map that is an empty box; the distance it printed at the end of these trials was between 0.00 and 0.20 m. The analyzer takes the ground-truth position. Table XI (from `experiments/campaign2/odom_gap.py`, a post hoc script outside the freeze) shows that in these trials the ground truth and the localization estimate disagree by 0.33 to 1.56 m and that the wheel odometry, shifted to the start pose, disagrees with the ground truth by 0.29 to 1.59 m. Wheel odometry and the AMCL estimate drift away from the ground truth in the SE pocket trials, after the turns and reversals of an escape. Table X extends the comparison to all 88 trials (post hoc, same script). We did not isolate the physical cause (wheel slip, the odometry plugin or another). The estimate does not simply follow the odometry: in the six C1 trials on s1 the AMCL estimate is 0.23 to 0.38 m from the ground truth while the odometry is at most 0.03 m from it. In the SE pocket trials the controller's output reached its limit (`wz_max` 1.9 rad/s) in 11 of 34 trials, all on s3; the command that reaches the base, after the velocity smoother, peaked at 1.23 to 1.90 rad/s. In the stock pocket trials both signals stay at or below 1.00 rad/s (C1) and 0.45 rad/s (C3). The pocket-exit time `left_pocket_s` is set in all 34 SE pocket trials (30 registered, 4 evidence) and in none of the 30 stock pocket trials. These trials count as not reached, so the counts in Table VIII are conservative with respect to Nav2's own report: counting Nav2's `SUCCEEDED` would give 6 of 6 for every SE-MPPI cell, which we do not claim. Deployment warning: the limits of this campaign (`wz_max` 1.9 rad/s; velocity smoother 2.0 rad/s and 3.2 rad/s^2) are those of the simulated robot, a robot with other limits needs its own measurement, and Nav2's success message after an escape should not be trusted without an independent check of the pose.

Teardown crash. One process died with a non-teardown exit code, in `s4_ltrap` C3 repetition 0, a stock-controller trial that had already ended in `NOT_REACHED_ABORTED`. The driver's teardown signal reached the processes 3.0 s after the drive ended and 4.0 s before the trial ended. The launch log then shows, in this order: the cleanup of the MPPI controller; the bond-timeout path of the localization lifecycle manager ("SERVER map_server IS DOWN", "Terminating bond

TABLE X
END OF THE DRIVE, ALL 88 TRIALS, BY CLASS: RANGE OF THE DISTANCE OF THE WHEEL ODOMETRY (SHIFTED TO THE START POSE) AND OF THE AMCL ESTIMATE FROM THE GROUND TRUTH (M), AND OF THE PEAK TURN RATE OF THE COMMAND THAT REACHES THE BASE, AFTER THE VELOCITY SMOOTHER (RAD/S). POST HOC; SOURCE: `EXPERIMENTS/CAMPAIGN2/ODOM_GAP.JSON`. THE SE CLASS INCLUDES THE 4 EVIDENCE TRIALS.

Class	Trials	Odometry	AMCL	Peak turn rate
SE-MPPI in a pocket	34	0.088–1.594	0.062–1.559	1.228–1.900
Stock in a pocket	30	0.001–0.131	0.003–0.375	0.039–1.000
`s0_open`, all	24	0.001–0.006	0.013–0.043	0.038–0.043

TABLE XI
THE 13 TRIALS WITH NAV2 `SUCCEEDED` AND A GROUND-TRUTH DISTANCE TO THE GOAL ABOVE 0.5 M: RANGE OVER THE 13 TRIALS OF THE END-OF-DRIVE POSITIONS (M). SOURCE: `EXPERIMENTS/CAMPAIGN2/ODOM_GAP.MD`.

Quantity	Minimum	Maximum
Ground-truth distance to goal	0.536	1.659
Distance printed by Nav2 (AMCL)	0.00	0.20
Shifted odometry distance to goal	0.013	0.728
Odometry vs ground truth	0.294	1.594
AMCL vs ground truth	0.328	1.559

timer"); a SIGSEGV message from the image library's signal handler, which carries no timestamp; and the exit of the composed container (component_container_isolated) with code -6. This is the only such death in the campaign, and the one on which amendment 3 was decided; that amendment classifies it a teardown crash, and RESULTS.md lists it per trial. We make no claim about its cause. On a robot, a crash of the composed container stops all navigation nodes that it hosts; use a respawn policy or a non-composed bringup.

VII. Limitations

- **Simulation only, one host, no real robot.** Every live result is a simulated robot in Gazebo on one workstation (WSL2, Ryzen 5 5500GT, 12 threads, simulated time; real-time factor 0.93 to 0.97). We ran no real robot and no second host. The claim is limited to the pre-registered wording: counts, Fisher p values and Wilson intervals, on one simulated host.
- **Six repetitions per cell.** The decisive contrasts are 5 of 6 against 0 of 6 ($p = 0.0152$), the smallest effect that clears the pre-registered level at this size; one more failure would have given $p = 0.0606$. The Wilson interval of 5 of 6 is $[0.44, 0.97]$. The comparison with Nav2's default recovery tree (C2 against C1: 3 of 6 against 0 of 6 on s1, 2 of 6 against 0 of 6 on s3) is not significant, so the live data do not establish an advantage inside the default tree. The seven contrasts use no correction for multiple tests; only two decide the branch.
- **The additions were found on the development family.** The four additions of Sec. IV-D were worked on until the 56 development geometries passed, and the advised design without (a) and (b) passes 14 to 18 of 56 in the three archived confirmation sweeps (19 in an earlier, unarchived sweep). The 56 of 56 is a development result. MPPI's noise cannot be seeded from the test, so each sweep is one independent process, and a geometry's outcome without the additions depended on its position in the sweep. The mirror benchmark without the additions gives a lower success on the trap families than with them (Table VI; on U-trap 58% against 80%, descriptive): the advised design already lifts U-trap success from 0%, and the additions add to it.
- **Held-out status.** s1_utrap is the development scenario, and static gap tests at its stall pose helped set the additions. s3_ctrap has a near mirror twin in the development family and is not decision-bearing; its result (4 of 6 against 0 of 6, $p = 0.0606$) is not significant. The claim of transfer rests on one pocket, s4_ltrap, which lies outside the family by depth (by 0.20 m, 11 percent) and shape but inside its span in width and rotation. One pocket shows that the controller escaped a geometry that the design did not see; it does not show that it does so for pockets in general.
- **Nav2's own report and the localization estimate.** 13 of 88 trials ended in Nav2 SUCCEEDED with the robot 0.54 to 1.66 m from the goal in the ground truth (Sec. VI-E1). Wheel odometry and the AMCL estimate drift after the turns and reversals of an escape, and the goal checker works from that estimate. We count these trials as not reached, which makes the live counts conservative, but a deployment that relies on the estimate would see the controller report success before the robot is at the goal. We did not isolate the cause. Reached trials drifted too: in the 21 SE pocket trials that reached the goal (19 registered and 2 evidence trials), the odometry is 0.09 to 0.54 m and the AMCL estimate 0.06 to 0.54 m from the ground truth at the end of the drive, so the reached count mixes escape with end-of-drive localization error.
- **The no-valid-control update cannot act under Nav2's default failure tolerance.** With failure_tolerance 0, as in all configurations of campaign 2, the controller server resets every controller when the optimizer throws, so the detector update on a throw (defect 2 of Sec. VI-A) acts only when the server tolerates the failure (a tolerance of -1, or a positive tolerance that has not yet run out, after which Jazzy's server aborts with PatienceExceeded). Campaign 2 did not exercise it, and its pass rule in the headless test requires zero throws.
- **Pre-registration is self-hosted.** The pre-registration was not deposited with a third party. The freeze dates can be checked only through the local commit history (ac0a4eb for the freeze, c6a144e and 012f4f1 for amendment 2, 6a50723 for amendment 3); the tag moved twice. The reader must trust that history.
- **Option B is out of scope.** The controller does not emit its own command when every rollout collides, for example when the footprint is already inside the inflated band. Then the optimizer throws and the detector runs. Only when the server tolerates the failure (a tolerance of -1, or a positive tolerance that has not yet run out) is an escape path handed to MPPI, in the next cycle; under the campaign's tolerance of 0 the server resets the controller in the same cycle. In either case MPPI cannot produce a command in the cycle that throws. No headless run and no campaign-2 trial threw, with reversing allowed.
- **The post hoc amendment.** Amendment 3 was written after a stopping rule fired once, and the crash it concerns happened in the shutdown of a stock-controller trial. It changes how that rule reads the evidence and departs from the rule of no source change after trial 1. Its effect is that the remaining trials ran, among them every C4 trial on s4, the pocket that carries the claim of transfer. The cause of the crash is not known. The crash scan also does not read the launch log of an interrupted or incomplete attempt, a gap older than the amendment.
- **Released defaults and loosened tolerances were not evaluated live.** The released parameter file uses 20 Hz, 56 steps of 0.05 s and batch 1000; the campaign used 10 Hz, 28 steps of 0.1 s and batch 1500. The stall window is 30 cycles in both, so it is 3 s in the campaign and 1.5 s at 20 Hz; the watchdog and the re-selection period are also counted in cycles, so all three stretch when the loop misses its rate. We did not evaluate the released defaults live; on this host a 20 Hz configuration with batch 2000 ran at about 5 Hz and lost the lifecycle bond (Sec. VI-B).

The campaign also loosened `transform_tolerance` (0.5 s) and `source_timeout` (5.0 s); do not copy them to a robot. The only `collision_monitor` polygon was disabled, so no `collision_monitor` zone was active in any live result.

- **Builds.** The live results come from the correct RoboStack build of the MPPI library (Table IV). Some distributed binaries roll trajectories out in the wrong direction; the July 2026 runs and 4 trials of campaign 1 used one, and we do not use them. The check the plugin carries covers the batch rollout integration, the known defect, and not every function of the library, and it guards only `SafeEscapeController`. Run `check_mppi_rollout` on every target install (apt binaries are untested), and never set `se_enforce_rollout_check` to false on a moving robot. Humble's distribution was not evaluated: every reported result comes from the Jazzy stack or the 2D mirror. The repository README records that the plugin builds on Humble behind a compile-time switch, that Humble's controller server never calls `reset()`, so a goal end does not clear the escape memory, and that the RoboStack Humble MPPI builds it checked roll out in the wrong direction.
- **Mirror versus controller.** The mirror transcribes the shipped escape, but it differs from the C++ controller in its progress signal with simulated replans, its analytic circular geometry, its cost, and in giving the CBF and the TTC the true obstacle states, so the costmap tracker is not exercised (Appendix). The 1,200 trials are 200 seeded worlds under six configs, not independent scenarios. Success is tested with paired McNemar tests, but the continuous metrics use unpaired Mann–Whitney tests conditioned on success, so they are descriptive.
- **Coordination benefit is unmeasured, not established.** The escape–safety coordination (C3) has the conditional invariance property of Sec. IV-F but produced no outcome difference over an independent escape+CBF stack in the mirror (Sec. VI-D1), and campaign 2 does not test it: its pockets hold no moving body. The tracker nevertheless admitted phantom obstacles in 42 of 46 SE-MPPI trials, including 8 of 12 on the trap-free `s0_open`, and the brake fallback forced stops on s3 and s4 only (up to 16 cycles in one trial; Sec. VI-E1). Whether admissions without a stop altered the commands was not logged. That was not a designed test of the coordination. Deployment warning: a robot can be stopped or deflected by obstacles that do not exist, in open space as well as in a pocket. The idea of adapting the CBF gain online is not new [28], [14], [30]. Demonstrating an outcome-level benefit requires a regime where the α choice decides which escapes are admitted.
- **Braking is not a safe fallback when cornered.** In the cramped narrow-dynamic pocket the infeasible-QP fallback ($v = 0$) leaves the robot in the path of a sweeping mover, and the CBF configs measured a *higher* collision rate than their no-CBF counterparts ($18\% \rightarrow 32\%$; Sec. VI-D). A cornered robot needs an evasive fallback, not a stop.
- **Look-ahead-point vs. full-body safety.** The Proposition of Sec. IV-F concerns the look-ahead point P, not the full robot body. Body safety is therefore **approximate**: the offset $L \approx r$ places P at the robot's leading edge, and the costmap obstacle critic covers the residual; in the live campaign no `collision_monitor` zone was active, so it covered nothing. A full-body guarantee would require inflating the margin by L, which we avoid because it over-constrains sub-meter gaps and would degrade the success rate. On the `dynamic` family the look-ahead filter left body-grazing collisions essentially unchanged ($20\% \rightarrow 16\%$, descriptive).
- **Sampled-data and slack caveats.** Invariance between samples is assumed in the Proposition, not proven, and the shared slack couples the rows of one solve; we report slack usage instead of claiming an unconditional guarantee. Static structure is outside the barrier, and in cycles with $\delta > \epsilon$ the braking fallback replaces the QP command.
- **Constant-velocity tracker.** The dynamic-obstacle model evaluated in this paper is a replaceable constant-velocity baseline; rotating or accelerating agents are mispredicted and prediction uncertainty is unquantified, and an obstacle that leaves the tracked, unpruned set $\mathcal{O}$ also leaves the barrier (Sec. IV-F). The controller that ran live carries an occupancy-based static/dynamic classification, persistent tracks with horizon prediction, and an online conformal bound that inflates the CBF effective radius. The campaign parameter file does not set `se_classify_static`, `se_predict_horizon` or `se_conformal`; their code defaults are true, so all three were on in every live SE trial. They are the subject of follow-up work (SE-Predict) and none is present in the mirror behind the quantitative results reported here. C4 and the tracker description of this paper hold for this configuration only.
- **Baselines and scale.** We ran no full-scale 3D physics benchmark (BARN, DynaBARN, HuNavSim [54], [55], [56]) and no run of DRPA-MPPI, GP-MPPI, a gap-based planner or an adaptive-gain CBF [12], [10], [22], [28]; we make no comparison to them. The only stack-level baseline is Nav2's default recovery tree around stock and SE-MPPI in campaign 2.
- **TTC, real time, localization.** TTC is a one-dimensional approximation; per-call QP cost grows with the obstacle budget (capped by clearance pruning); the tracker stamps a costmap frame in the control cycle that first sees it, so its velocity estimates carry the jitter of that rounding and, on a robot, the costmap latency; localization in symmetric or narrow maps can jump and perturb progress estimation.

VIII. Conclusion

SE-MPPI is a system that packages local-minima escape and control-barrier-function safety, two problems that several prior works also address together outside Nav2, in a single Nav2 controller plugin: a shared entrapment signal drives an

escape that hands MPPI a path to an opening and an output-time CBF filter, and a coordinated gain raises the barrier's class-$\mathcal{K}$ gain during escape, which, by a standard argument for time-varying gains, keeps the look-ahead point's safe set with respect to the tracked dynamic obstacles forward invariant, assuming the barrier inequality holds between control updates and the QP uses no slack.

The first version did not reach the goal live: 0 of 44 valid trials on a correct build. Four defects explain it (one is inferred, not logged), and the repaired controller carries four additions to the advised escape, two needed together for the development sweeps to pass (ablated jointly, confounded with position in the sweep) and two safeguards for other configurations; all four are disclosed, with a joint ablation of the first two. In the pre-registered campaign on one simulated host (one post hoc amendment to its crash-stop rule, after which all s4 SE-MPPI trials ran), SE-MPPI without recoveries reached the goal in 5 of 6 on s1 (Wilson 95% interval $[0.44, 0.97]$) and 5 of 6 on the held-out s4; stock without recoveries in 0 of 6 on each; with Nav2's default recovery tree, on s1, stock 0 of 6 and SE-MPPI 3 of 6. The Fisher exact p of the two decisive contrasts is 0.0152. s1 is the development scenario; s4 carries the claim. The third pocket, s3, gave 4 of 6 against 0 of 6 ($p = 0.0606$), which is not significant. The result does not rest on a real robot, a second host or more than six repetitions per cell.

A randomized, paired $1{,}200$-trial benchmark on a Python 2D mirror of the shipped algorithm shows the escape raising success on the U-trap family from 0% to 80%, and to 58% without the four additions, while the escape–safety coordination is indistinguishable from an independent escape+CBF stack there, which we report as a null. We addressed three integration faults that surfaced in the live stack – static-wall CBF infeasibility (the wall freeze stopped; phantom admissions remained), host-matched compute budgeting, and an xtensor ABI mismatch – and we guard against a defective distributed MPPI binary. The remaining steps are a real robot, more repetitions and pockets, a test of the coordination in a regime where it can matter, and a full-scale physics benchmark against the baselines of Sec. II. The codebase also contains a prediction stack and a multi-robot layer that are the subject of separate work and are not evaluated here; the prediction features ran at their code defaults in campaign 2 (Sec. VII).

APPENDIX A
REPRODUCIBILITY

All paths below are relative to the code release of contribution C5 (footnote in Sec. I). The commits and tags below are local to the repository at the time of writing.

Software. ROS 2 Jazzy, Nav2, and Gazebo are installed via RoboStack (conda channels `robostack-jazzy` + `conda-forge`) by the committed script `scripts/setup_ros2_env.sh`, which also pulls `osqp-eigen/eigen` for the CBF-QP [47] and the toolchain. A new environment must resolve to the `jazzy_25` package set, whose `nav2_mppi_controller` is 1.3.13 build `_25`; `ros2 run nav2_se_controller check_mppi_rollout` must print OK. The package list in `experiments/results_pilot_rerun/env_correct.txt` is the campaign-1 snapshot of the environment that campaign 2 also used (same environment name in each trial's `meta.json`); each campaign-2 trial records only the key builds and the `check_mppi_rollout` result in its `meta.json`. The Python 2D mirror, the benchmark, its report and the figure script need no ROS; their environment is `experiments/benchmark2d/environment.yml` (Python 3.12.13, numpy 2.4.6, osqp 1.1.3, scipy 1.18.0, matplotlib 3.11.0).

Tags and commits. The commits, the tag and the raw-data archive named here are local to the author's repository at the time of writing and are not published (the tag was never pushed). A release ref will be published before submission; the commit of the local tag `campaign2-controller` is the provenance. The controller of the live campaign is that commit. Trials 1 to 16 of campaign 2 ran at `012f4f1` (the tag before amendment 3), the later ones at `6a50723`; `src/`, `experiments/sim` and the campaign worlds are identical at the two commits. The confirmation sweeps of Table III ran at `b7e012a`, whose controller sources are those of `507173a`; against the tag, `src/` at `b7e012a` differs in one comment of the test rig. The full hashes of every commit named in this paper are below. The mirror re-run ran at `742a3df` (the commit that also added the campaign-2 analysis outputs); its results are in `0388854`. Run the commands below at the release ref: the runner, the mirror and the report are unchanged since `742a3df`, while the figure script (legend layout and a command-line entry), the fallback-threshold script and the environment file come from `11b30b7`. The design-stage benchmark of Appendix B, with its figures, was generated at the tag `paper-v1`; the current figure script does not rebuild those figures byte for byte, because their legend layout changed.

```
campaign2-controller
  6a5072324d2ca60785049279588b5bd7204c86ea
mirror re-run (code)
  742a3df16d2d9f0adb617b9c020dc64962b2dfe1
mirror re-run (results)
  0388854d74d6880002972b5932eafd7a44abb8d7
paper-v1
  b21b0a546f36804a2af7d44e0f2e98ba1c235cac
campaign trials 1-16 (tag before amendment 3)
  012f4f1db11de8ce607854578cb99ab932bf3969
confirmation sweeps
  b7e012a7473ad1cf3014c4a7ab53afccb16a8661
controller sources of the sweeps
  507173a2a45bef81020029ece9078278ee39ad18
figure script, fallback script, environment
  11b30b78e06ff36eece8bb0d162e1d52e37aae7f
pre-registration freeze
  ac0a4ebfe908fda24da21cf7a98dd1ddc227ea9c
amendment 2
  c6a144ee1d080805f2f349cf71c76e818a2a90aa
```

Mirror re-run (both arms). Environment: `experiments/benchmark2d/environment.yml`. The trial results are deterministic. The runner resumes by default and skips scenarios already in its output file, so a rerun needs `--no-resume` and a fresh output directory; the commands below write to `/tmp` and compare with the committed files.

```
for arm in v2 v2_advised; do
  python3 -m experiments.benchmark2d.runner --variant $arm \
    --families utrap clutter dynamic narrowdyn -n 50 \
    --workers 6 --max-steps 400 --no-resume \
    --out /tmp/mirror_$arm/trials.csv
  python3 -m experiments.benchmark2d.report \
    --trials /tmp/mirror_$arm/trials.csv \
    --out-dir /tmp/mirror_$arm
  python3 -m experiments.benchmark2d.compare_v1_v2 \
    /tmp/mirror_$arm/trials.csv \
    > /tmp/mirror_$arm/COMPARE_V1.md
  # committed: experiments/results_2d_$arm
  sort /tmp/mirror_$arm/trials.csv > /tmp/new.csv
  sort experiments/results_2d_$arm/trials.csv > /tmp/old.csv
  cmp /tmp/new.csv /tmp/old.csv && echo "$arm: identical"
done
python3 -m experiments.benchmark2d.figures \
  --trials experiments/results_2d_v2/trials.csv \
  --out-dir docs/papers/latex/figures \
  --suffix _v2
B=experiments.benchmark2d
A=experiments/results_2d_v2_advised
python3 -m $B.advised_fallback_threshold \
  -n 50 > $A/FALLBACK_THRESHOLD.md
```

The runner refuses to write either arm to `experiments/results_2d/trials.csv`. The determinism check regenerates the design-stage variant (`--variant v1`) over all 1,200 trials; every row equals a row of the committed `experiments/results_2d/trials.csv` (row order differs, because the pool returns scenarios in completion order); the note is `experiments/results_2d_v2_smoke/FULL_RERUN.md`. Each arm's directory holds `trials.csv`, `tables.md`, `stats.json`, `summary.csv` and `COMPARE_V1.md` (written by `compare_v1_v2`, as above). The figures `success_bars_v2.png` and `e_vs_f_v2.png` are written by the `figures` command above from the shipped arm's `trials.csv`; the last command prints `FALLBACK_THRESHOLD.md` of the advised arm (Sec. IV-D) to standard output, redirected here over the committed file, which `git diff` then compares.

Headless development test. `test/deployment/test_closed_loop_utrap.cpp`, with the family rule in `test/deployment/UTRAP_DEV_GEOMETRIES.md`; the sweep over all 56 valid geometries runs with `SE_UTRAP_SWEEP=all`. The additions are switched off with `SE_UTRAP_PARAMS`; the values need a decimal point, because the rig reads `0` as an integer. In the ROS environment, from the repository root (the commands name the test and the variables as the build files define them; they were not rerun for this revision):

```
P=nav2_se_controller
colcon build --packages-select $P
source install/setup.bash
H="--event-handlers console_direct+"
T="--ctest-args -R test_closed_loop_utrap"
# additions on
SE_UTRAP_SWEEP=all \
  colcon test --packages-select $P $H $T
# additions off
OFF=se_escape_avoid_radius=0.0
OFF=$OFF,se_escape_avoid_hop_radius=0.0
OFF=$OFF,se_escape_stall_time=1e6
SE_UTRAP_SWEEP=all SE_UTRAP_PARAMS=$OFF \
  colcon test --packages-select $P $H $T
```

One sweep is one process; the sweeps cannot be seeded. A sweep can hang in the teardown of the test rig after its last row (`PREREG.md`, lines 225–234); ctest's `TIMEOUT` of 1200 s then fails the test although the rows are complete. The rows of the confirmation-2 sweeps that Table III quotes are in `experiments/campaign2/sweeps/`.

Mirror–C++ parity test. The `colcon build` above (with `BUILD_TESTING`, the default) also builds `parity_shim`. Then, from the repository root, in the mirror environment:

```
SE_PARITY_SHIM=build/nav2_se_controller/parity_shim \
SE_REQUIRE_PARITY=1 PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 \
  python3 -m pytest \
  experiments/benchmark2d/tests/test_v2_cxx_parity.py
```

Run it with `SE_REQUIRE_PARITY=1`: without it the test skips silently when the shim is missing, and a "pass" then means that nothing was compared.

Live campaign 2. From the repository root, in the campaign environment with the workspace sourced:

```
C=experiments/campaign2
python3 $C/run_campaign2.py --list
python3 $C/run_campaign2.py --block A
python3 $C/run_campaign2.py --block B
python3 $C/analyze.py --raw RAW --out OUT --pack
python3 $C/analyze.py --raw RAW --check
python3 $C/odom_gap.py --raw RAW --out-dir OUT
export PYTEST_DISABLE_PLUGIN_AUTOLOAD=1
python3 -m pytest $C/tests
```

The raw trial files are the archive `experiments/campaign2/raw_trials.tar.gz` (sha256 in the file next to it): 88 trial directories (84 registered trials, namely 24 on `s0_open`, 24 on `s1_utrap`, 24 on `s3_ctrap` and 12 on `s4_ltrap`, and 4 evidence trials on `s1_utrap_evidence`), the 5 entries of the shakedown directory and the driver logs. Unpack it into an empty directory and pass its `campaign2_raw` directory as `RAW`. Without `--raw` the analysis reads the original raw root `/mnt/d/wsl/se-mppi/campaign2`. Without `--out`, `analyze.py` overwrites the committed `results.json.gz`, `RESULTS.md` and `logs/`; `--check` only reads. `odom_gap.py` also writes its two output files into the repository directory by default, so pass `--out-dir` to write elsewhere. The other committed analysis outputs are `results.json.gz`, `RESULTS.md` and the log excerpts in `experiments/campaign2/logs/`. `analyze.py --check` compares them with a fresh analysis of the raw data (`--raw RAW`) and passes only in the environment that wrote them: another Python can differ in the last digit of some floats, while `RESULTS.md` is byte-identical in both. `PREREG.md` holds the pre-registration with its amendments, `SHAKEDOWN.md` the one trial on `s0_open` that counts for nothing, and `FREEZE_MANIFEST.json` the hashes of the frozen files.

Number guard. `python3 scripts/check_paper_numbers.py` checks the numbers of this paper against the committed artifacts: the mirror arms (`stats.json`, `summary.csv`), the campaign (`results.json.gz`, `ODOM_GAP.json`, the wording of `PREREG.md`), campaign 1 (`results.json`), the design-stage benchmark of Appendix B, the test counts and the parameter file. It also checks the prose counts of the abstract, of the introduction (C2) and of the conclusion

against `results.json.gz`, the per-class drift table against `ODOM_GAP.json`, the forced-stop counts, the raw archive with its hash, and the committed sweep rows.

2D mirror versus the Nav2 controller. The 2D mirror (`experiments/prototype/se_mppi_v2.py` for the shipped escape, driven by `experiments/benchmark2d/rollout.py`) follows the controller's formulas but differs in the following documented ways.

- *Progress signal.* Progress is the arc length along an anchor path, with straight replans simulated once a second, as in the controller; the design-stage mirror of Appendix B used the best distance to the goal instead.
- *Geometry.* Obstacles, including walls built from chains of discs, are circles; repulsion distances and ray lengths are computed analytically against them rather than on a costmap distance field. The repulsion has the same Khatib form but other parameters and distances: $d_0 = 0.8$ m and $\eta = 0.3$, the distance to the surface of each disc (clamped at 0.02 m), and the sum over discs of the per-disc horizon mean; the controller uses $d_0 = 0.6$ m, $\eta = 1$ and the distance to the nearest inscribed-band cell, without a sum.
- *Obstacle knowledge.* The CBF and TTC receive the true positions and velocities of the moving obstacles, so the tracker of Sec. IV-G is not exercised.
- *Costs.* The MPPI cost is a goal-distance, path-progress, and obstacle-proximity cost rather than Nav2's critic stack, and the parameters differ from the released controller's.
- *Stack.* Nav2's progress checker and behavior-tree recoveries are not modeled.

Data and parameters. The full parameter set of the live campaign is `experiments/sim/nav2_se_campaign2.yaml`; the released defaults are in `src/nav2_se_controller/config/nav2_se_controller_params.yaml`.

APPENDIX B
DESIGN-STAGE BENCHMARK (PAPER-V1)

An earlier version of this paper reported a 1,200-trial benchmark and a mechanism validation run on a Python 2D mirror of a different, design-stage algorithm: detection from the best distance to the goal, an opening chosen among rays with 2.4 m of 3.0 m free, and a subgoal 2.5 m along the opening that replaced the goal while entrapped. Sec. VI-A explains why that algorithm is not the plugin. We keep the results here, labelled as the design stage; they are not claims about the shipped controller. The data are `experiments/results_2d/` at the tag `paper-v1`; the mirror that produced them is unchanged and regenerates them exactly. The setup (families, seeds, configs, tests) is that of Sec. VI-D.

A. Randomized benchmark of the design stage

Tables XII and XIII and Figs. 5 and 6 give the design-stage results. In the design-stage mirror the escape layer lifts success from 0% to 88–90% on the `utrap` family (44/50 discordant pairs, Holm-adjusted $p = 1.4 \times 10^{-9}$) and to 62–78% on `narrowdyn` (31/50 discordant, $p_{\text{adj}} = 1.1 \times 10^{-6}$); the no-gap config F$^-$ scores 0% on both, so the free-space gap subgoal was load-bearing there. The coordination contrast is null in every family (McNemar $p = 1$), as in the shipped arm. The re-run on the mirror of the shipped algorithm (Sec. VI-D) gives lower success on the trap families: this is the number the paper reports.

TABLE XII
DESIGN-STAGE BENCHMARK ON THE PYTHON 2D MIRROR OF THE FIRST VERSION (NOT THE NAV2 CONTROLLER AND NOT THE SHIPPED ALGORITHM): SUCCESS RATE IN % (WILSON 95% CI), $N = 50$ PER CELL.

Config	U-trap	Clutter	Dynamic	Narrow-dyn
A stock	0 [0,7]	52 [39,65]	80 [67,89]	0 [0,7]
C escape	90 [79,96]	62 [48,74]	76 [63,86]	78 [65,87]
D CBF	0 [0,7]	52 [39,65]	82 [69,90]	0 [0,7]
E indep.	88 [76,94]	62 [48,74]	78 [65,87]	64 [50,76]
F SE-MPPI	88 [76,94]	62 [48,74]	78 [65,87]	62 [48,74]
F$^-$ no-gap	0 [0,7]	52 [39,65]	84 [71,92]	0 [0,7]

TABLE XIII
KEY PAIRED CONTRASTS OF THE DESIGN-STAGE BENCHMARK (MCNEMAR, HOLM-ADJUSTED; b/c DISCORDANT PAIRS).

	Escape (stock→F)		Coord. (E→F)	
Family	b/c	p_{adj}	b/c	p_{adj}
U-trap	0/44	1.4×10^{-9} *	0/0	1
Clutter	0/5	1	0/0	1
Dynamic	3/2	1	0/0	1
Narrow-dyn	0/31	1.1×10^{-6} *	1/0	1

B. Mechanism validation (standalone 2D)

To validate each mechanism independently of simulator infrastructure, we reimplemented the controller's core math (repulsion, gap ray cast, look-ahead CBF-QP via OSQP [47], α modulation + TTC override, monotone-progress entrapment) as a 2D unicycle and ran three scenarios. This code is the design-stage mirror; in these three scenes its CBF sees all obstacles, including the static wall discs. Sources: `experiments/prototype/`, generated by `experiments/prototype/run_validation.py`. Table XIV gives the measurements. Three findings: (1) stock MPPI stalls in front of a U-trap while SE-MPPI detects and rounds it; (2) against a crossing dynamic obstacle, the no-CBF baseline collides while the speed-aware CBF filter avoids it; and (3) the coordination plot shows the gain rise $\alpha : 2 \to 6$ during the escape phase with slack ≈ 0 throughout, close to the slack-free setting that the Proposition of Sec. IV-F assumes. These are shown in Fig. 7.

REFERENCES

[1] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A Navigation System," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. [Online]. Available: https://arxiv.org/abs/2003.00368

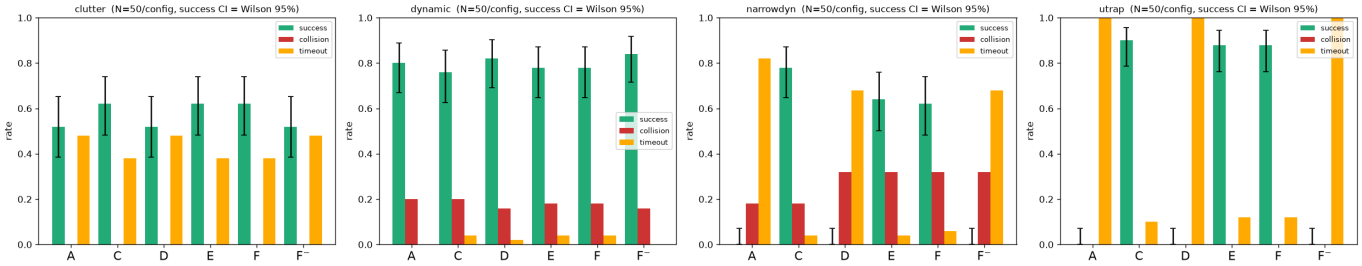


Fig. 5. Design-stage benchmark: success / collision / timeout per family across configs A–F$^-$ (Wilson 95% error bars on success, $N = 50$).

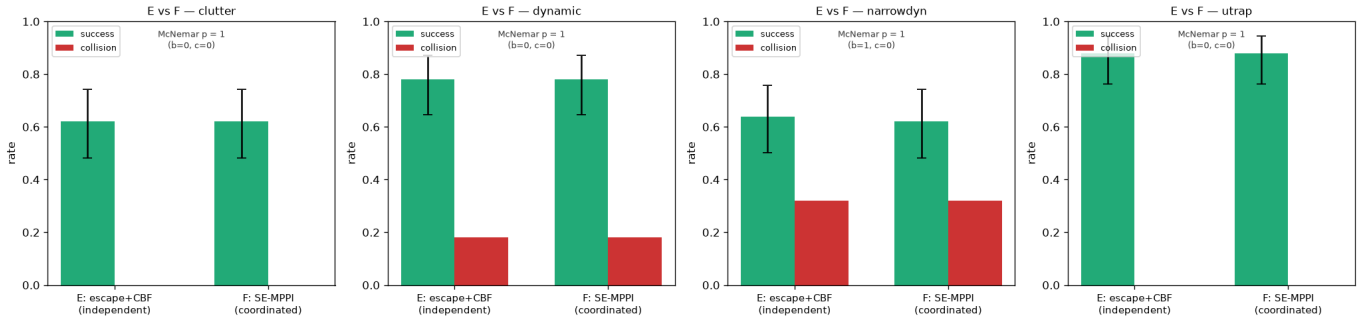


Fig. 6. The coordination contrast E vs. F per family in the design-stage benchmark, annotated with the McNemar p-value.

TABLE XIV
STANDALONE 2D MECHANISM VALIDATION OF THE DESIGN STAGE (MEASURED).

Scenario / config	reached	collided	time	min-clear (m)
U-trap / Stock MPPI	× (stuck)	—	—	0.33
U-trap / SE-MPPI (escape)	✓	—	27.7 s	0.32
Dynamic / No CBF	×	collision	—	−0.00
Dynamic / SE-MPPI (CBF)	✓	safe	18.6 s	0.01
Coordination: SE-MPPI vs. independent†	✓	safe	27.7 s	0.32

†SE-MPPI (F) and the independent stack (E) return identical trajectories and metrics in this benign scenario.

[2] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving," 2017. [Online]. Available: https://arxiv.org/abs/1707.02342

[3] Nav2 contributors, "nav2_controller, branch jazzy: controller_server.cpp and simple_progress_checker.cpp," Nav2 source code, GitHub repository ros-navigation/navigation2, 2026, accessed 2026-09-30. [Online]. Available: https://github.com/ros-navigation/navigation2/tree/jazzy/nav2_controller

[4] ——, "Detailed Behavior Tree Walkthrough," Nav2 documentation, 2026, accessed 2026-09-30. [Online]. Available: https://docs.nav2.org/rolling/getting_started/nav2_behavior_trees/detailed_behavior_tree_walkthrough/detailed_behavior_tree_walkthrough/

[5] I. S. Mohamed, K. Yin, and L. Liu, "Autonomous Navigation of AGVs in Unknown Cluttered Environments: log-MPPI Control Strategy," *IEEE Robotics and Automation Letters*, 2022. [Online]. Available: https://arxiv.org/abs/2203.16599

[6] E. Trevisan and J. Alonso-Mora, "Biased-MPPI: Informing Sampling-Based Model Predictive Control by Fusing Ancillary Controllers," *IEEE Robotics and Automation Letters*, 2024, arXiv:2401.09241. [Online]. Available: https://arxiv.org/abs/2401.09241

[7] J. Sacks and B. Boots, "Learning Sampling Distributions for Model Predictive Control," in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, 2022. [Online]. Available: https://arxiv.org/abs/2212.02587

[8] Z. Wang, O. So, J. Gibson, B. Vlahov, M. S. Gandhi, G.-H. Liu, and E. A. Theodorou, "Variational Inference MPC using Tsallis Divergence," 2021. [Online]. Available: https://arxiv.org/abs/2104.00241

[9] K. Honda, N. Akai, K. Suzuki, M. Aoki, H. Hosogaya, H. Okuda, and T. Suzuki, "Stein Variational Guided Model Predictive Path Integral Control: Proposal and Experiments with Fast Maneuvering Vehicles," 2023. [Online]. Available: https://arxiv.org/abs/2309.11040

[10] I. S. Mohamed, M. Ali, and L. Liu, "GP-Guided MPPI for Efficient Navigation in Complex Unknown Cluttered Environments," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 7463–7470. [Online]. Available: https://arxiv.org/abs/2307.04019

[11] T. Fuke, M. Endo, K. Honda, and G. Ishigami, "Towards Local Minima-free Robotic Navigation: Model Predictive Path Integral Control via Repulsive Potential Augmentation," in *2025 IEEE/SICE International Symposium on System Integration (SII)*, 2025, pp. 1112–1117. [Online]. Available: https://arxiv.org/abs/2410.11379

[12] ——, "DRPA-MPPI: Dynamic Repulsive Potential Augmented MPPI for Reactive Navigation in Unstructured Environments," 2025. [Online]. Available: https://arxiv.org/abs/2503.20134

[13] J. Yin, C. Dawson, C. Fan, and P. Tsiotras, "Shield Model Predictive Path Integral: A Computationally Efficient Robust MPC Approach Using Control Barrier Functions," 2023. [Online]. Available: https://arxiv.org/abs/2302.11719

[14] H. Parwana, T. Kim, K. Long, B. Hoxha, H. Okamoto, G. Fainekos, and D. Panagou, "BR-MPPI: Barrier-Rate Guided MPPI for Enforcing Multiple Inequality Constraints with Learned Signed Distance Fields," 2025. [Online]. Available: https://arxiv.org/abs/2506.07325

[15] H. Parwana, M. Black, G. Fainekos, B. Hoxha, H. Okamoto, and D. Prokhorov, "Model Predictive Path Integral Methods with Reach-Avoid Tasks and Control Barrier Functions," 2024. [Online]. Available: https://arxiv.org/abs/2407.13693

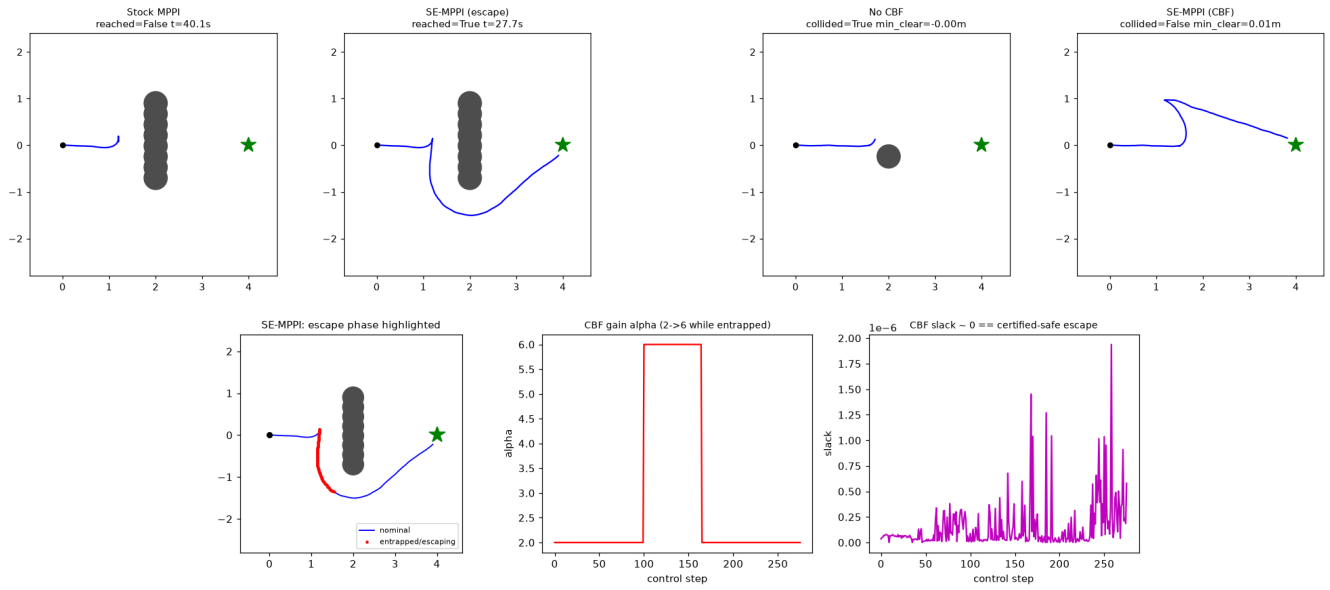


Fig. 7. Standalone 2D mechanism validation of the design stage (trajectories/traces only; no benchmark metrics). **(a)** U-trap escape: stock MPPI (left) stalls at the trap mouth while SE-MPPI (right) detects the stalled progress and rounds the U-shaped obstacle to the goal via the gap-attraction subgoal. **(b)** Dynamic-obstacle avoidance: the no-CBF baseline (left) collides while the speed-aware look-ahead CBF filter (right) projects the command to a collision-free path. **(c)** Escape–safety coordination: the coordinated gain rises $\alpha : 2 \to 6$ while the QP slack stays ≈ 0.

[16] J. Borquez, L. Raus, Y. U. Ciftci, and S. Bansal, "DualGuard MPPI: Safe and Performant Optimal Control by Combining Sampling-Based MPC and Hamilton-Jacobi Reachability," *IEEE Robotics and Automation Letters*, vol. 10, no. 7, 2025. [Online]. Available: https://arxiv.org/abs/2502.01924

[17] P. Rabiee and J. B. Hoagg, "Guaranteed-Safe MPPI Through Composite Control Barrier Functions for Efficient Sampling in Multi-Constrained Robotic Systems," 2024. [Online]. Available: https://arxiv.org/abs/2410.02154

[18] Nav2 contributors, "Enhanced Safety for Nav2 using 3Laws Supervisor," Nav2 documentation, 2026, accessed 2026-09-30. [Online]. Available: https://docs.nav2.org/rolling/tutorials/general_tutorials/using_3laws_supervisor/using_3laws_supervisor/

[19] ACCESS Newswire, "Open Navigation and 3Laws Demonstrate Capabilities of Nav2 and 3Laws Supervisor Integration," Press release, 18 December 2025, as syndicated by The Globe and Mail, 2025, accessed 2026-09-30. [Online]. Available: https://www.theglobeandmail.com/investing/markets/markets-news/ACCESS%20Newswire/36691510/open-navigation-and-3laws-demonstrate-capabilities-of-nav2-and-3laws-supervisor-integration/

[20] R. Soloperto, A. Mesbah, and F. Allgöwer, "Safe Exploration and Escape Local Minima With Model Predictive Control Under Partially Unknown Constraints," *IEEE Transactions on Automatic Control*, vol. 68, no. 12, pp. 7530–7545, 2023. [Online]. Available: https://arxiv.org/abs/2205.03614

[21] R. Xu, S. Feng, and P. A. Vela, "Potential Gap: Using Reactive Policies to Guarantee Safe Navigation," arXiv preprint arXiv:2103.11491, 2021. [Online]. Available: https://arxiv.org/abs/2103.11491

[22] S. Feng, A. Abuaish, and P. A. Vela, "Safer Gap: A Gap-based Local Planner for Safe Navigation with Nonholonomic Mobile Robots," arXiv preprint arXiv:2303.08243, 2023. [Online]. Available: https://arxiv.org/abs/2303.08243

[23] M. Asselmeier, D. Ahuja, A. Zaro, A. Abuaish, Y. Zhao, and P. A. Vela, "Dynamic Gap: Safe Gap-based Navigation in Dynamic Environments," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 12 870–12 876. [Online]. Available: https://arxiv.org/abs/2210.05022

[24] Y. Xue and N. Figueroa, "No Minima, No Collisions: Combining Modulation and Control Barrier Function Strategies for Feasible Dynamic Collision Avoidance," arXiv preprint arXiv:2502.14238, 2025. [Online]. Available: https://arxiv.org/abs/2502.14238

[25] M. F. Reis, A. P. Aguiar, and P. Tabuada, "Control Barrier Function-Based Quadratic Programs Introduce Undesirable Asymptotically Stable Equilibria," *IEEE Control Systems Letters*, vol. 5, no. 2, pp. 731–736, 2021. [Online]. Available: https://doi.org/10.1109/LCSYS.2020.3004797

[26] Y. Xue, Z. Zhang, K. Åkesson, and N. Figueroa, "Proactive Local-Minima-Free Robot Navigation: Blending Motion Prediction With Safe Control," *IEEE Robotics and Automation Letters*, vol. 11, no. 4, pp. 5096–5103, 2026. [Online]. Available: https://arxiv.org/abs/2601.10233

[27] Y.-H. Chen, S. Ghori, A. Adil, E. Feron, and C. Belta, "Long-Horizon Geometry-Aware Navigation among Polytopes via MILP-MPC and Minkowski-Based CBFs," arXiv preprint arXiv:2604.00162, 2026. [Online]. Available: https://arxiv.org/abs/2604.00162

[28] T. Kim, R. I. Kee, and D. Panagou, "Learning to Refine Input Constrained Control Barrier Functions via Uncertainty-Aware Online Parameter Adaptation," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 3868–3875. [Online]. Available: https://arxiv.org/abs/2409.14616

[29] W. Xiao, C. Belta, and C. G. Cassandras, "Adaptive Control Barrier Functions for Safety-Critical Systems," arXiv preprint arXiv:2002.04577, 2020. [Online]. Available: https://arxiv.org/abs/2002.04577

[30] H. Parwana and D. Panagou, "Rate-Tunable Control Barrier Functions: Methods and Algorithms for Online Adaptation," in *2025 American Control Conference (ACC)*, 2025, pp. 275–282. [Online]. Available: https://arxiv.org/abs/2303.12966

[31] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, "Trajectory Modification Considering Dynamic Constraints of Autonomous Robots," in *Proceedings of ROBOTIK 2012; 7th German Conference on Robotics*. VDE Verlag, 2012, pp. 74–79. [Online]. Available: https://ieeexplore.ieee.org/document/6309484

[32] D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.

[33] S. Macenski, S. Singh, F. Martin, and J. Gines, "Regulated Pure Pursuit for Robot Path Tracking," *Autonomous Robots*, 2023. [Online]. Available: https://arxiv.org/abs/2305.20026

[34] O. Khatib, "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986. [Online]. Available: https://doi.org/10.1177/027836498600500106

[35] M. Black, G. Fainekos, B. Hoxha, H. Okamoto, and D. Prokhorov, "CBFKit: A Control Barrier Function Toolbox for Robotics Applications," 2024. [Online]. Available: https://arxiv.org/abs/2404.07158

[36] H. K. Park, T. Kim, and D. Panagou, "Beyond Collision Cones: Dynamic Obstacle Avoidance for Nonholonomic Robots via Dynamic Parabolic Control Barrier Functions," 2025, accepted to IEEE International Conference on Robotics and Automation (ICRA) 2026. [Online]. Available: https://arxiv.org/abs/2510.01402
[37] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control Barrier Function Based Quadratic Programs for Safety Critical Systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017. [Online]. Available: https://doi.org/10.1109/TAC.2016.2638961
[38] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control Barrier Functions: Theory and Applications," in *2019 18th European Control Conference (ECC)*, 2019, pp. 3420–3431. [Online]. Available: https://arxiv.org/abs/1903.11199
[39] J. Zeng, B. Zhang, Z. Li, and K. Sreenath, "Safety-Critical Control using Optimal-decay Control Barrier Function with Guaranteed Point-wise Feasibility," in *2021 American Control Conference (ACC)*, 2021, pp. 3856–3863. [Online]. Available: https://arxiv.org/abs/2103.12375
[40] R. Walia and K. Leahy, "Conflict-Aware Switching for CBF-CLF-Based Multi-Goal Navigation," arXiv preprint arXiv:2606.21577, 2026, cDC 2026 submission copy. [Online]. Available: https://arxiv.org/abs/2606.21577
[41] H. Paudel, B. Joshi, D. R. Madai, A. Bartaula, B. Rimal, and S. Neupane, "A Closed-Form Dual-Barrier CBF Safety Filter for Holonomic Robots on Incrementally Built Occupancy Grid Maps," arXiv preprint arXiv:2605.05182, 2026. [Online]. Available: https://arxiv.org/abs/2605.05182
[42] H. Zhou, Y. Zhang, and W. Luo, "Safety-Critical Control with Uncertainty Quantification using Adaptive Conformal Prediction," in *2024 American Control Conference (ACC)*, 2024. [Online]. Available: https://arxiv.org/abs/2407.03569
[43] J. Gonzales, K. Mizuta, K. Leung, and L. J. Ratliff, "Safe Probabilistic Planning for Human-Robot Interaction using Conformal Risk Control," 2026. [Online]. Available: https://arxiv.org/abs/2603.10392
[44] L. Lindemann, M. Cleaveland, G. Shim, and G. J. Pappas, "Safe Planning in Dynamic Environments using Conformal Prediction," 2022. [Online]. Available: https://arxiv.org/abs/2210.10254
[45] A. Dixit, L. Lindemann, S. Wei, M. Cleaveland, G. J. Pappas, and J. W. Burdick, "Adaptive Conformal Prediction for Motion Planning among Dynamic Agents," 2022. [Online]. Available: https://arxiv.org/abs/2212.00278
[46] L. Busellato, F. Cunico, D. Dall'Alba, M. Emporio, A. Giachetti, R. Muradore, and M. Cristani, "Uncertainty Aware-Predictive Control Barrier Functions: Safer Human Robot Interaction through Probabilistic Motion Forecasting," 2025. [Online]. Available: https://arxiv.org/abs/2508.20812
[47] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An Operator Splitting Solver for Quadratic Programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020. [Online]. Available: https://arxiv.org/abs/1711.08013
[48] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927. [Online]. Available: https://doi.org/10.1080/01621459.1927.10502953
[49] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. [Online]. Available: https://doi.org/10.1007/BF02295996
[50] H. B. Mann and D. R. Whitney, "On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947. [Online]. Available: https://doi.org/10.1214/aoms/1177730491
[51] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993. [Online]. Available: https://doi.org/10.1037/0033-2909.114.3.494
[52] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733
[53] R. A. Fisher, "On the Interpretation of χ^2 from Contingency Tables, and the Calculation of P," *Journal of the Royal Statistical Society*, vol. 85, no. 1, pp. 87–94, 1922. [Online]. Available: https://doi.org/10.2307/2340521
[54] D. Perille, A. Truong, X. Xiao, and P. Stone, "Benchmarking Metric Ground Navigation," 2020. [Online]. Available: https://arxiv.org/abs/2008.13315
[55] A. Nair, F. Jiang, K. Hou, Z. Xu, S. Li, X. Xiao, and P. Stone, "DynaBARN: Benchmarking Metric Ground Navigation in Dynamic Environments," in *2022 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. IEEE, 2022, pp. 347–352.
[56] N. Pérez-Higueras, R. Otero, F. Caballero, and L. Merino, "HuNavSim: A ROS 2 Human Navigation Simulator for Benchmarking Human-Aware Robot Navigation," 2023, accepted for publication in IEEE Robotics and Automation Letters (RA-L). [Online]. Available: https://arxiv.org/abs/2305.01303